

Deep Learning for Computer Vision (Generative way)

Andrii Liubonko
2025

Course structure

Module 1. Basic image processing - Oles Dobosevych

Module 2. Learning Discriminative Models - Andrii Liubonko

Module 3. Image Segmentation, Image Correspondence - Maksym Davydov

Module 4. Metric Learning, Representation Learning and Context Importance, Tracking - Igor Krashenyi

Module 5. Learning Generative Models - Andrii Liubonko

Overview of this module

Lecture I, Lecture II

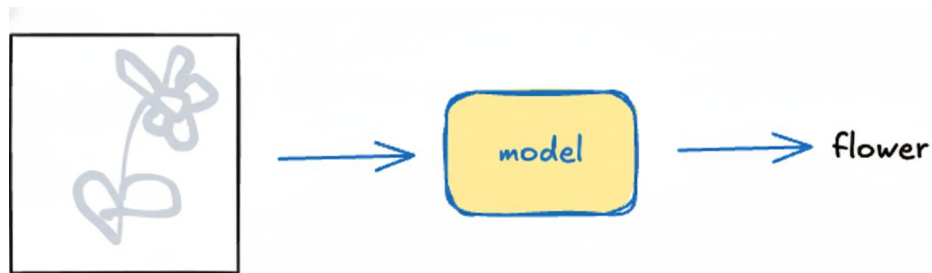
- Intro to generative modeling
 - Diffusion models
 - Stable diffusion
-

Lecture III, Lecture IV

- 3D / Geometric Data
- 3D Understanding
- 3D Generation

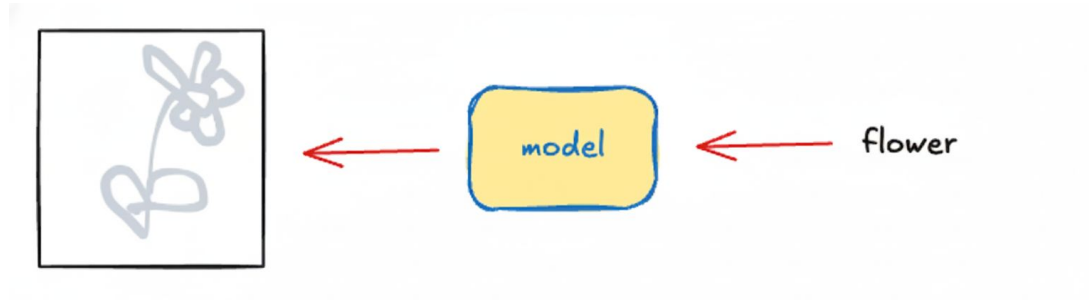
Quick recap

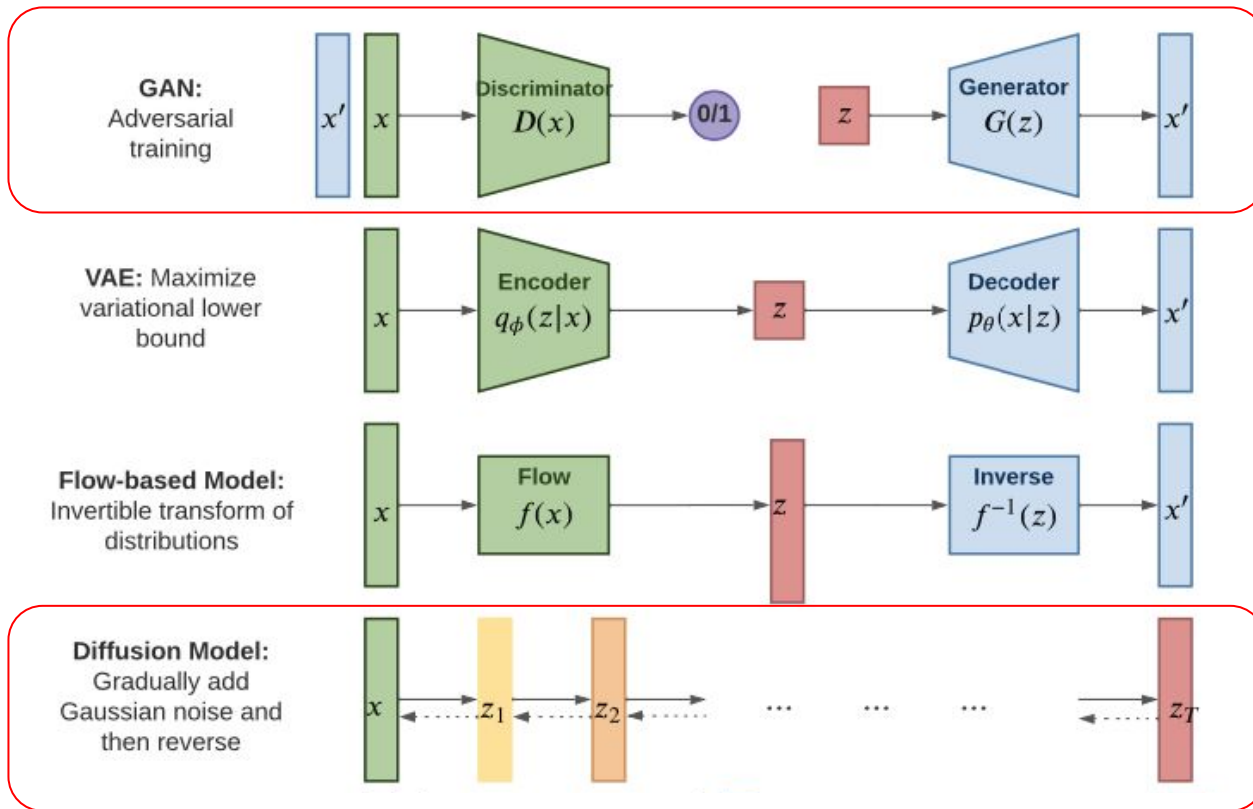
→ to extract useful information from visual input **(Discriminative Models)**

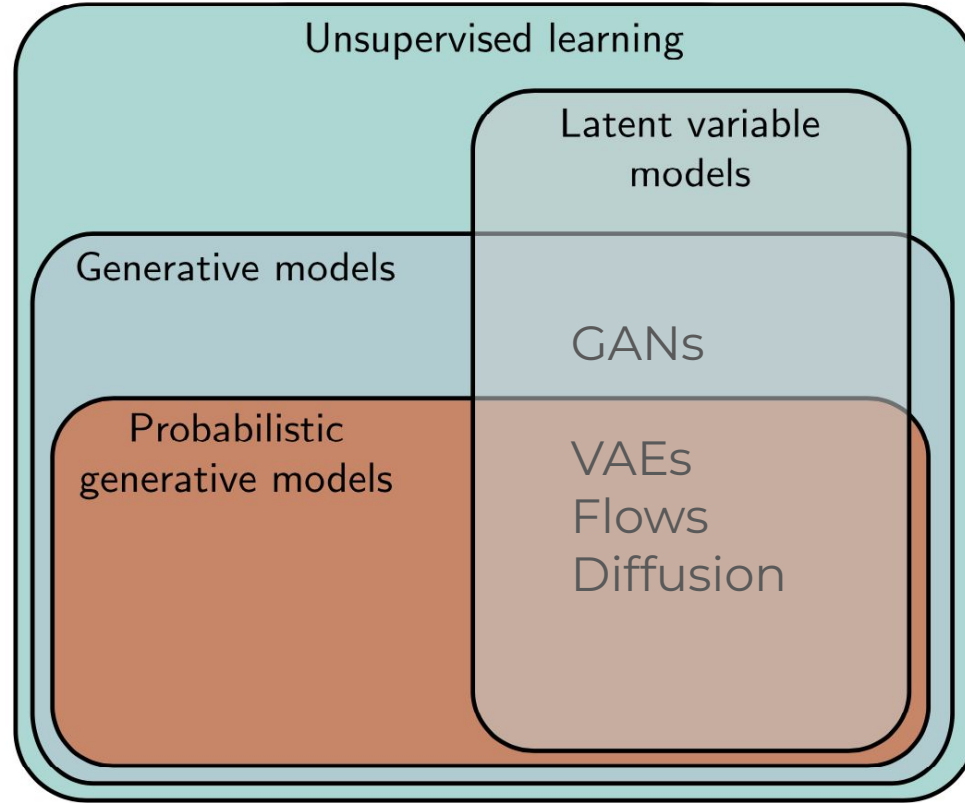


→ to generate novel visual content

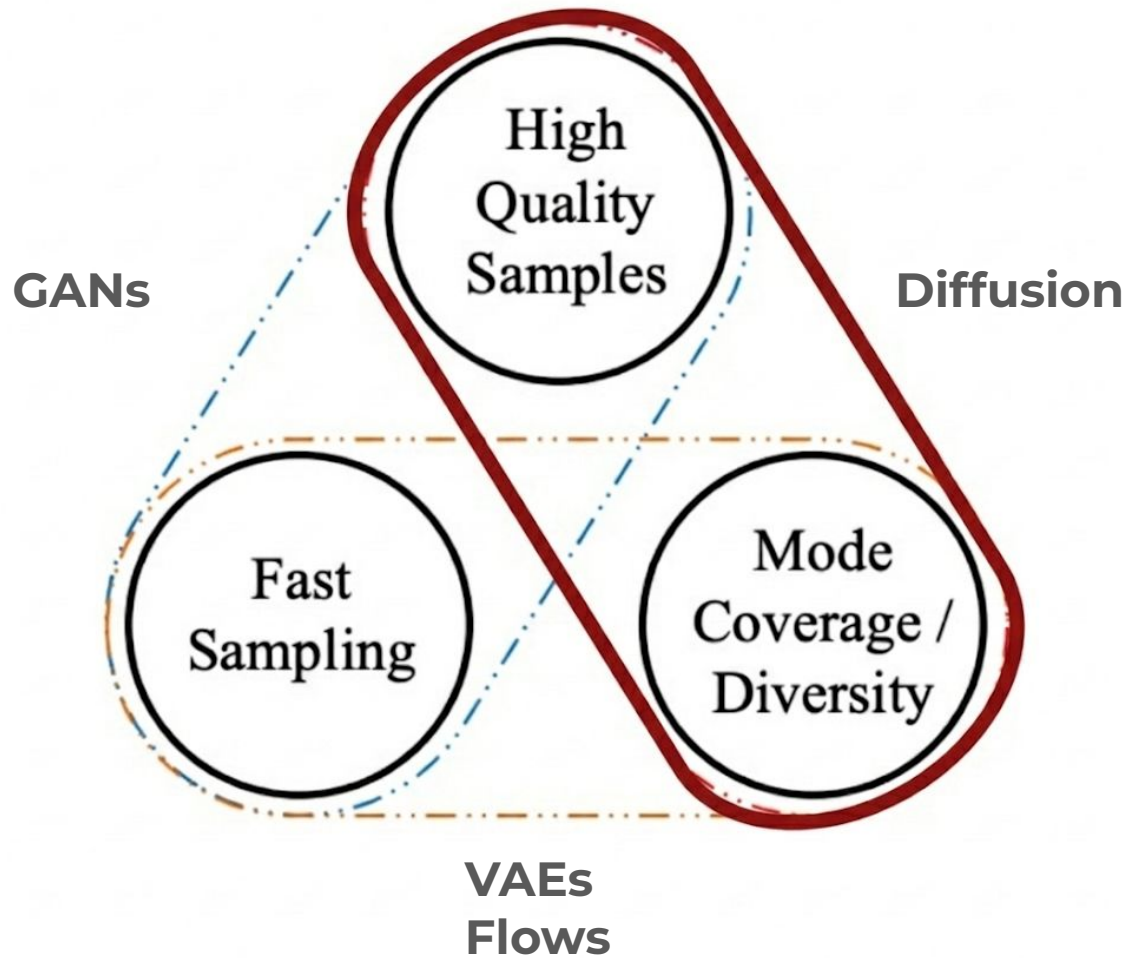
(Generative Models)



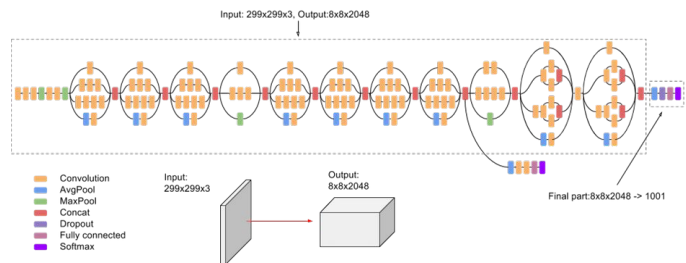




* image from [Understanding Deep Learning](#), book by Simon J.D. Prince, 2023



Metrics [Fréchet Inception Score vs. Inception score]



Feature	Inception Score (IS)	Fréchet Inception Distance (FID)
Goal	Measure Clarity & Diversity	Measure Realism (Distance to Ground Truth)
Reference Data	None (Uses pre-trained ImageNet knowledge)	Requires Real Data (Compares Real vs. Fake)
Mechanism	Class Probabilities (Softmax Output)	Feature Embeddings (Internal Layer)
Direction	Higher is Better	Lower is Better
Pros	Good for checking if objects are recognizable.	Captures texture/realism better; sensitive to mode collapse.
Cons	Can't measure similarity to a specific dataset.	Computationally expensive; requires large sample size.

Generative adversarial network (GAN)

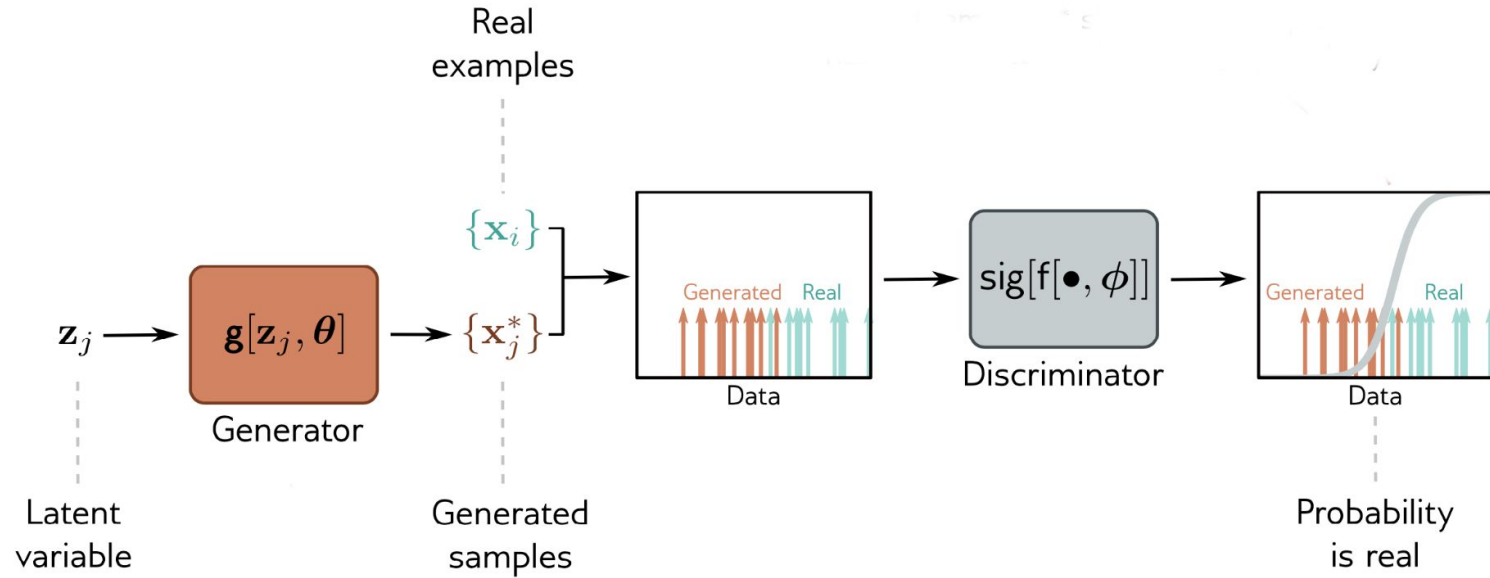


Image translation

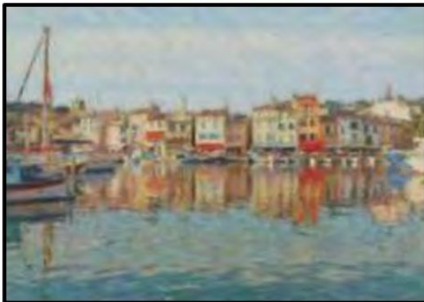
Horse

Zebra



Photo

Monet



Zebra

Horse



Monet

Photo



Super-resolution

b)

Bicubic (4×)



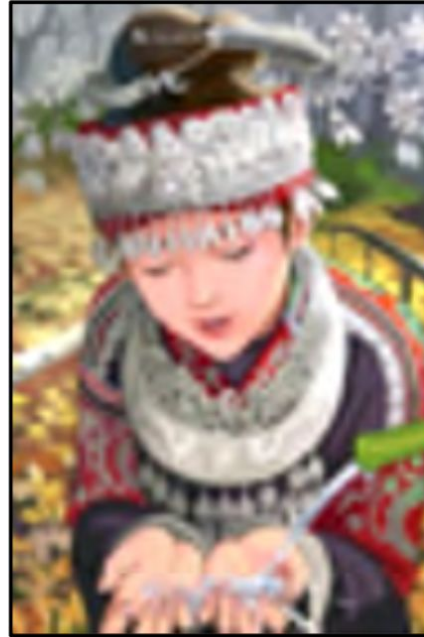
c)

SRGAN (4×)



d)

Bicubic (4×)



e)

SRGAN (4×)



[Photo-realistic single image super-resolution using a generative adversarial network](#)

Diffusion Model

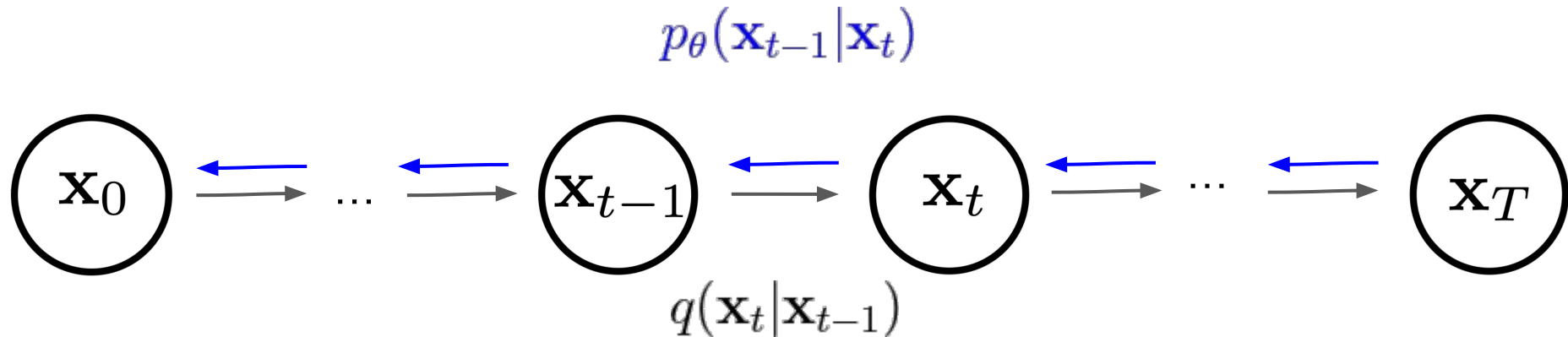
“The sculpture is already complete within the marble block, before I start my work. It is already there, I just have to chisel away the superfluous material.”

© Michelangelo



Structure

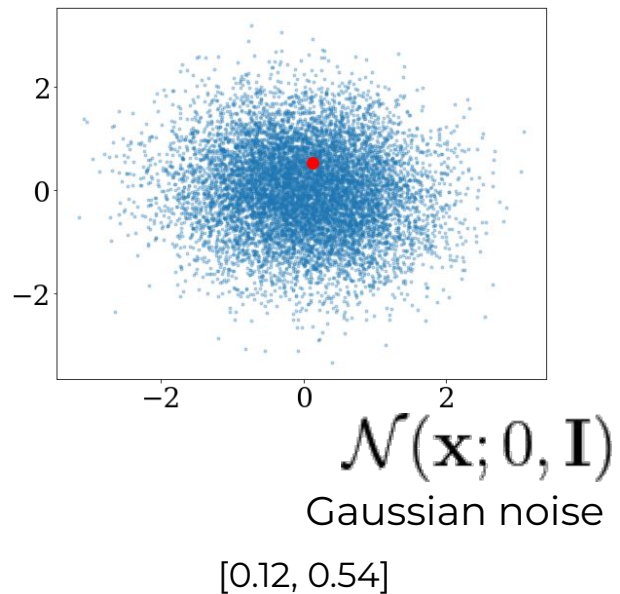
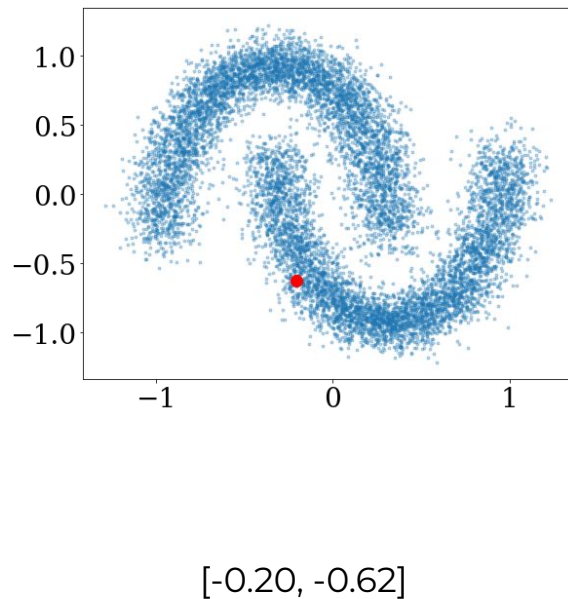
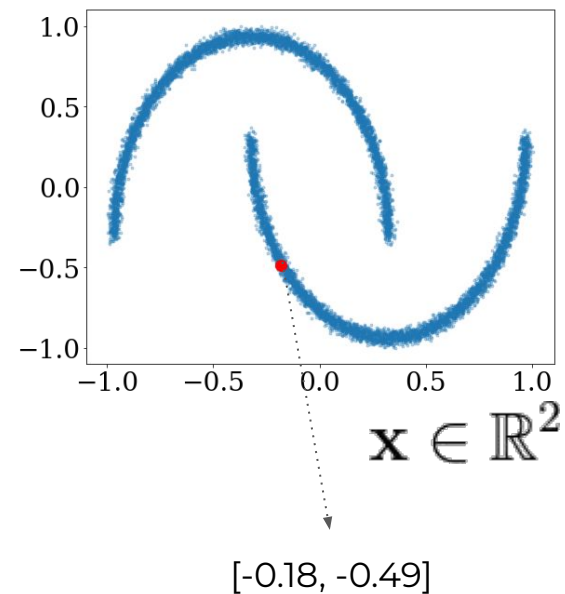
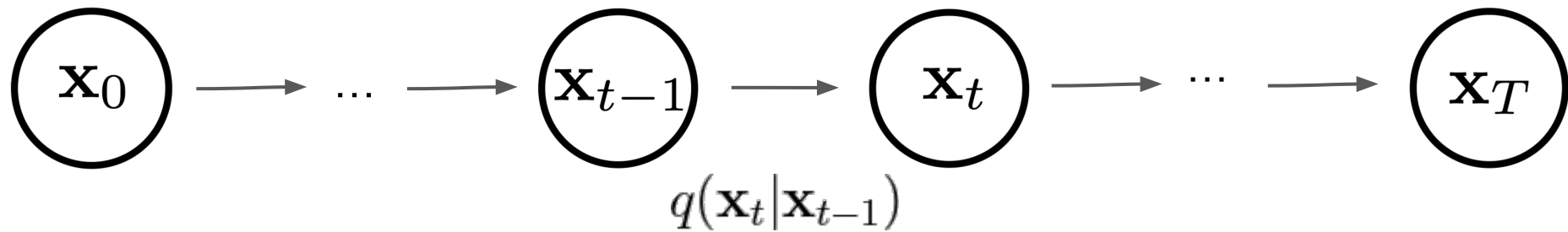
Uniform
distribution



$$\begin{aligned}
 p_\theta(\mathbf{x}_{t-1}|\mathbf{x}_t) &= \mathcal{N}(\mathbf{x}_{t-1}; \mu_\theta(\mathbf{x}_t, t), \Sigma_\theta(\mathbf{x}_t, t)) \\
 &= \mathcal{N}(\mathbf{x}_{t-1}; \mu_\theta(\mathbf{x}_t, t), \sigma_t^2 \mathbf{I})
 \end{aligned}$$

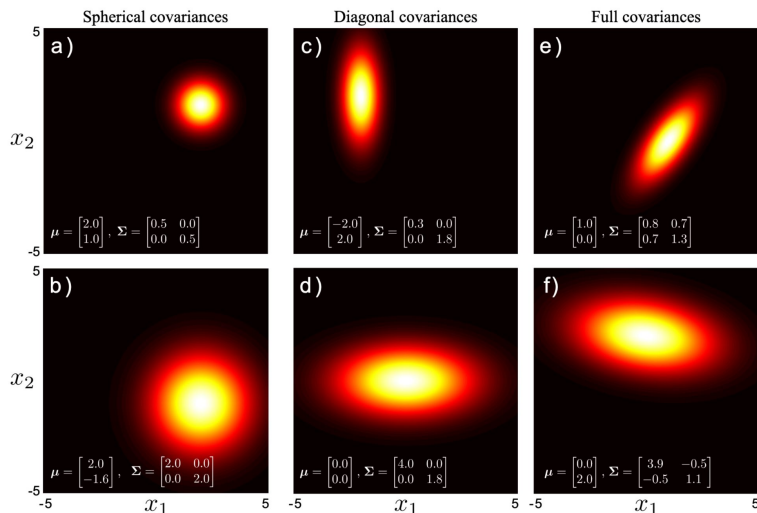
$$\mu_\theta(\mathbf{x}_t, t) = \frac{1}{\alpha_t} \left(\mathbf{x}_t - \frac{\beta_t}{\sqrt{1 - \bar{\alpha}_t}} \cdot \underbrace{\epsilon(\mathbf{x}_t, t)}_{\text{Trainable network}} \right)$$

Trainable network



Multivariate Gaussian distribution

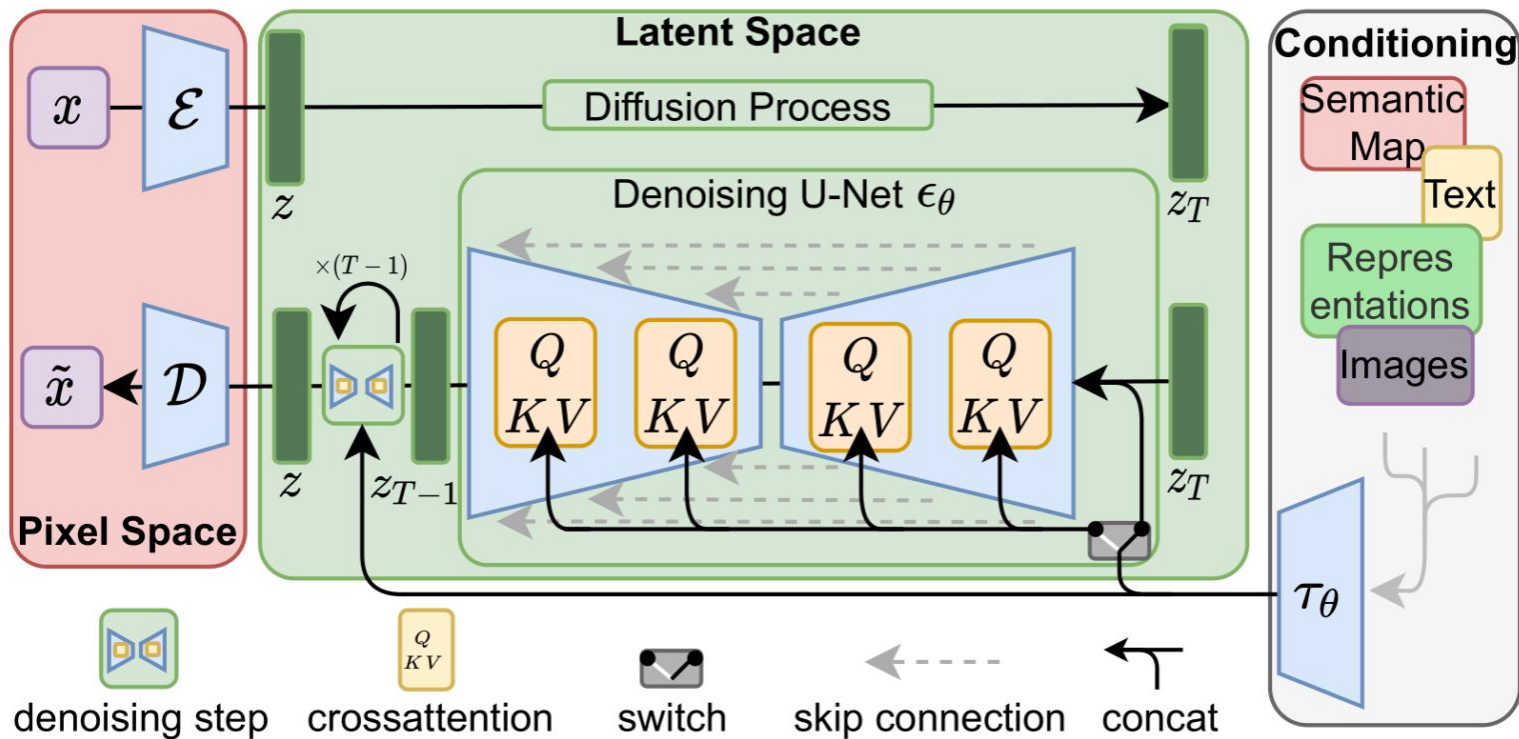
$$\mathcal{N}(\mathbf{x}; \boldsymbol{\mu}, \boldsymbol{\Sigma}) = \frac{1}{|\boldsymbol{\Sigma}|^{1/2} (2\pi)^{D/2}} \exp \left[-\frac{1}{2} (\mathbf{x} - \boldsymbol{\mu})^T \cdot \boldsymbol{\Sigma}^{-1} \cdot (\mathbf{x} - \boldsymbol{\mu}) \right]$$



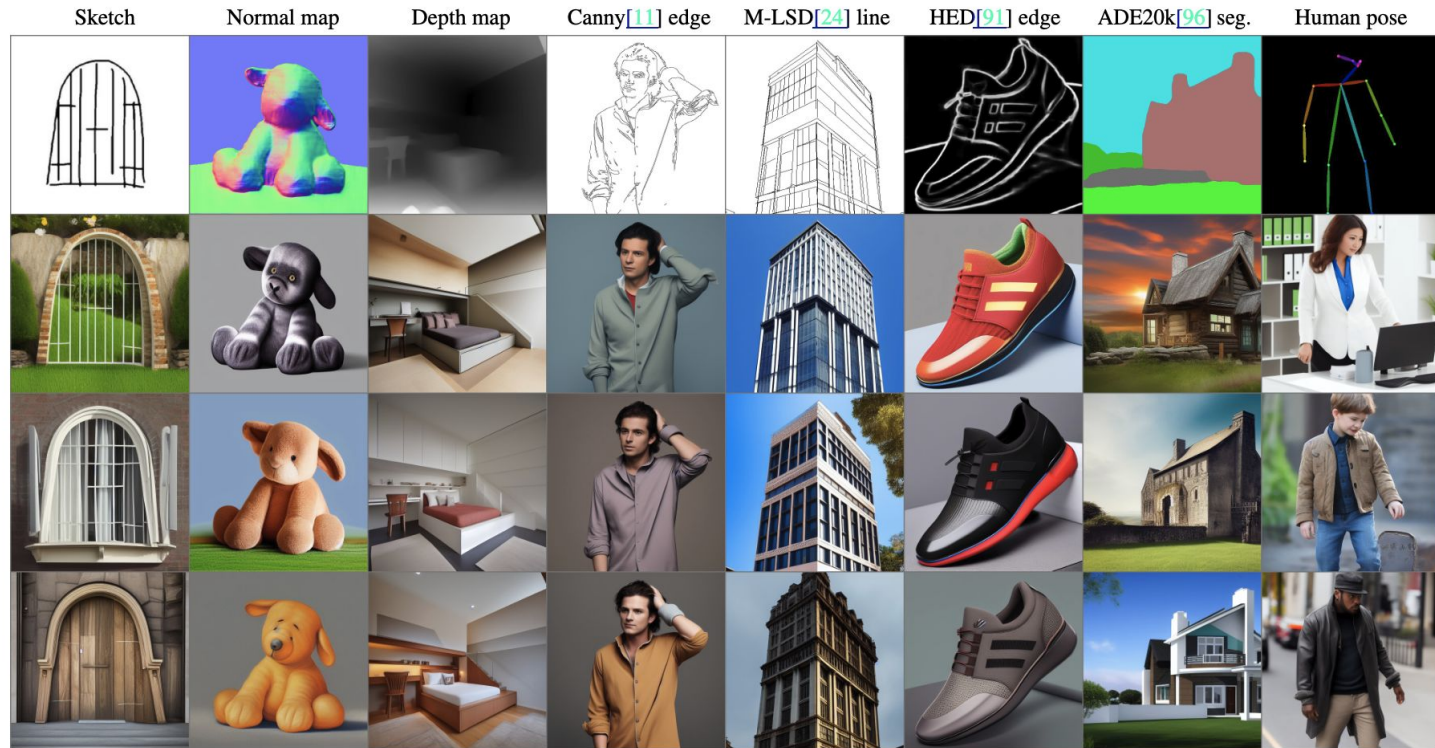
$\boldsymbol{\mu}$ is $[D \times 1]$ vector that describes the position of the distribution

$\boldsymbol{\Sigma}$ is $[D \times D]$ positive definite matrix (implying that $\mathbf{z}^T \cdot \boldsymbol{\Sigma} \cdot \mathbf{z}$ is positive for any real vector $\mathbf{z}$) and describes the shape of the distribution

Stable Diffusion



[\[2112.10752\] High-Resolution Image Synthesis with Latent Diffusion Models](#)

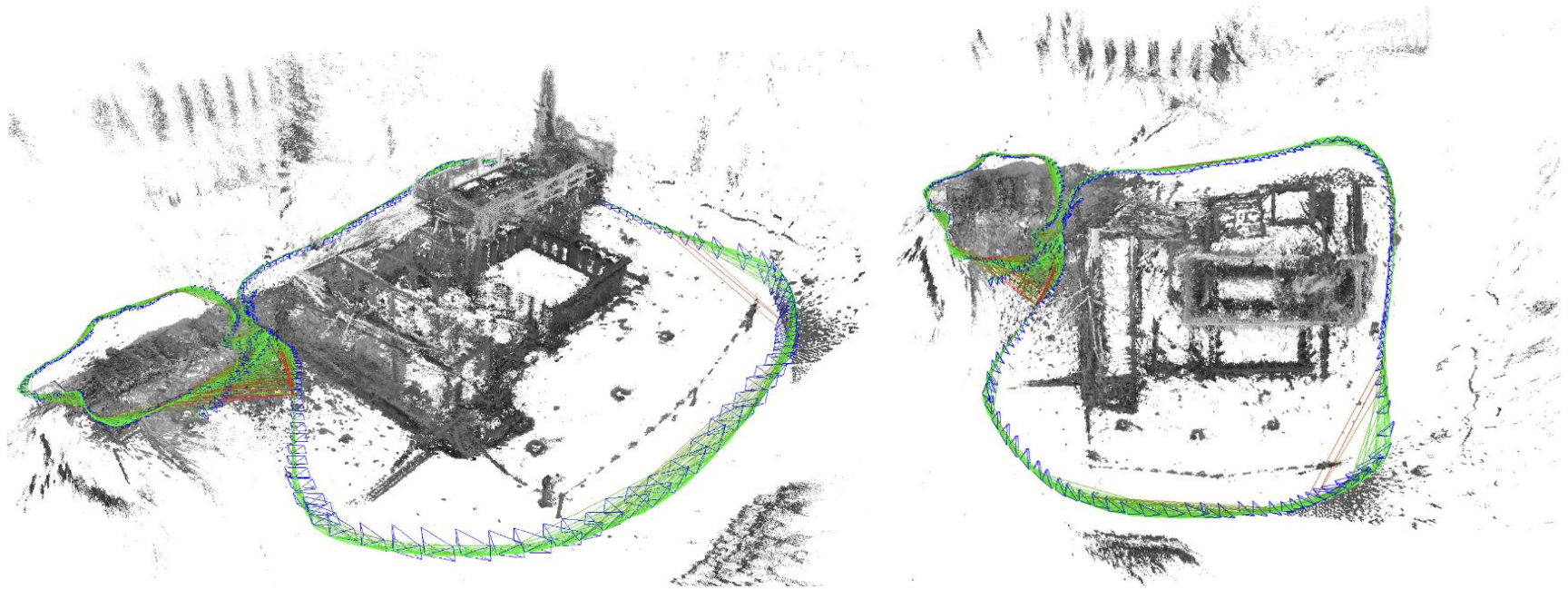


[2302.05543] Adding Conditional Control to Text-to-Image Diffusion Models

Content of today's lecture

- 3D / Geometric Data
 - Intro
 - 3D representations
 - Processing
- 3D Perception / Understanding
- 3D Generation

3D perception (SLAM)

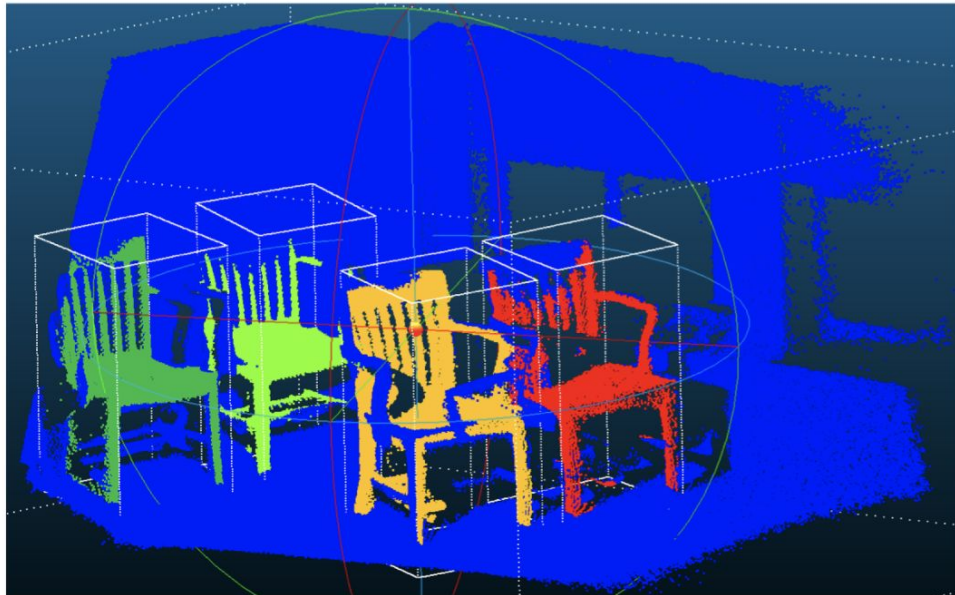
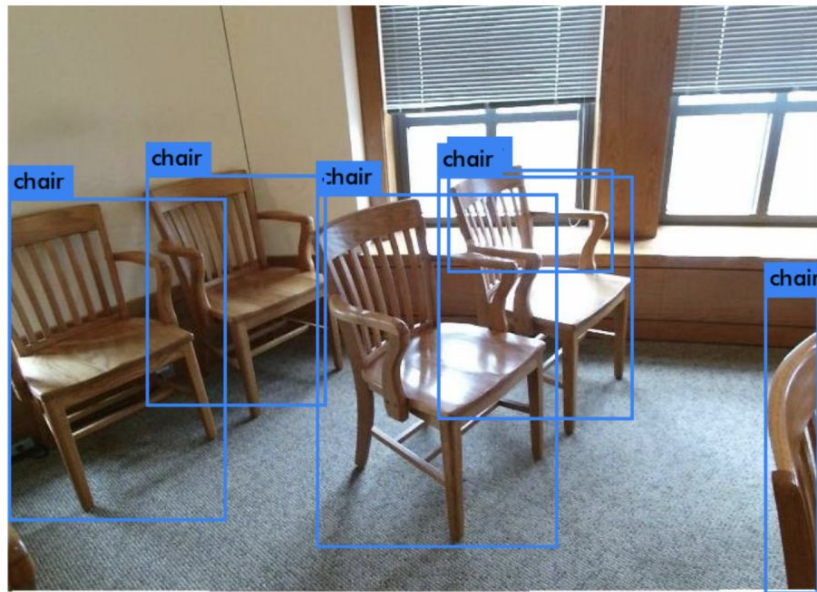


[LSD-SLAM: Large-scale direct monocular SLAM](#)

3D reconstruction (2D => 3D)



3D detection/segmentation

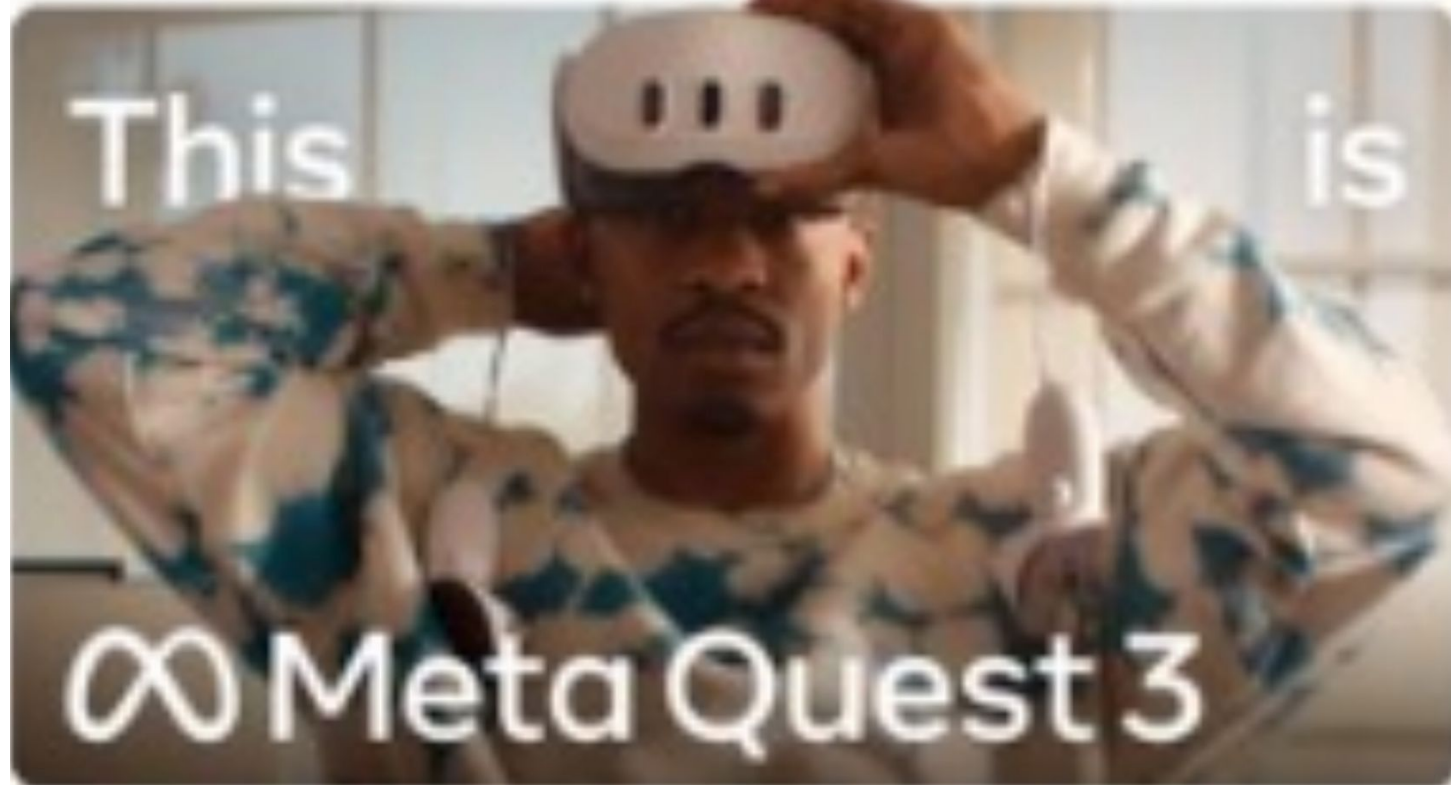


3D generative models



<https://huggingface.co/spaces/stabilityai/stable-fast-3d>

Virtual reality / Augmented reality



Content of today's lecture

- 3D / Geometric Data
 - Intro
 - 3D representations
 - Processing
- 3D Perception / Understanding
- 3D Generation

3D representation

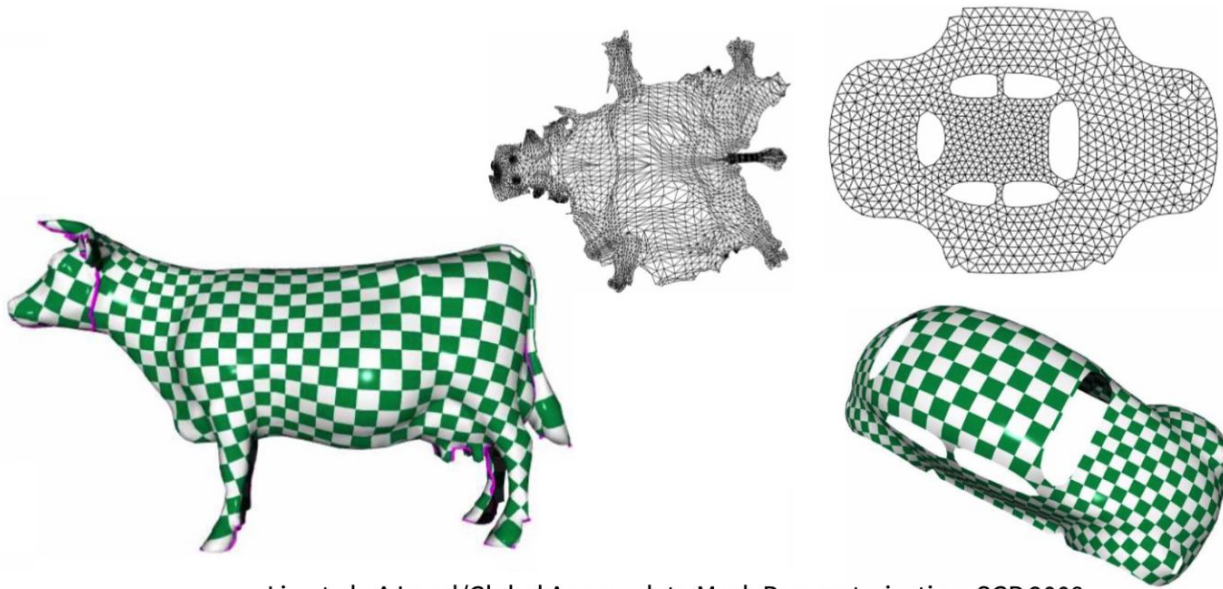


what humans see

0	3	2	5	4	7	6	9	8
3	0	1	2	3	4	5	6	7
2	1	0	3	2	5	4	7	6
5	2	3	0	1	2	3	4	5
4	3	2	1	0	3	2	5	4
7	4	5	2	3	0	1	2	3
6	5	4	3	2	1	0	3	2
9	6	7	4	5	2	3	0	1
8	7	6	5	4	3	2	1	0

what computers see

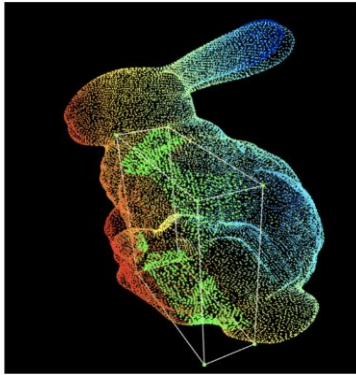
3D representation



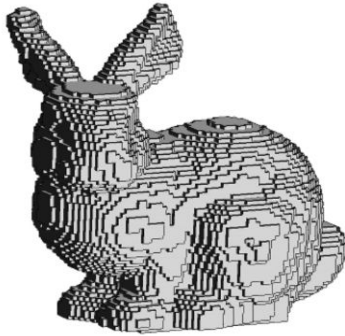
Liu et al., A Local/Global Approach to Mesh Parameterization, SGP 2008.

We are interested in 2d surfaces in 3D space

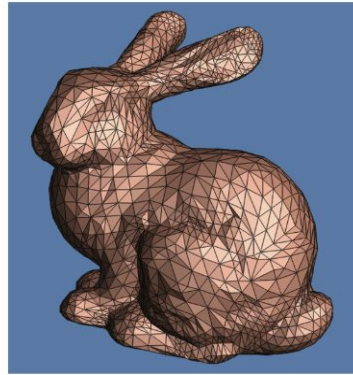
3D representation



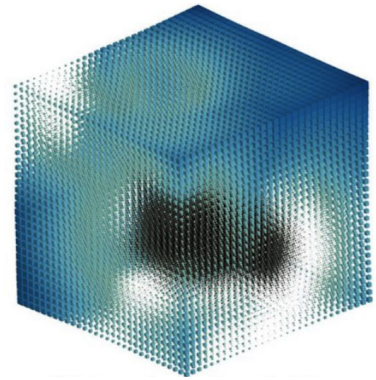
point cloud



voxel grid



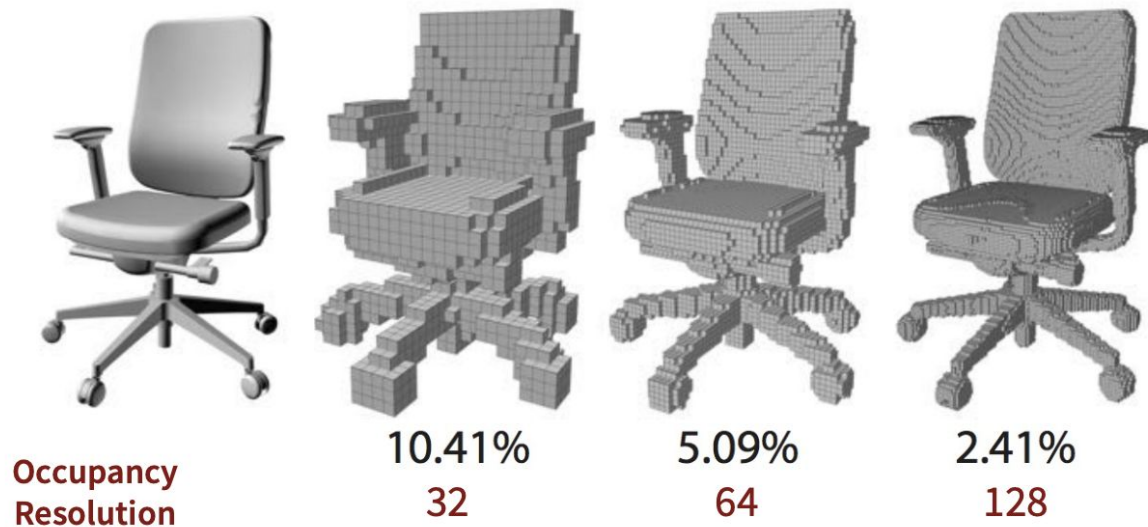
mesh



implicit

<https://thegradients.pub/beyond-the-pixel-plane-sensing-and-learning-in-3d/>

Voxel



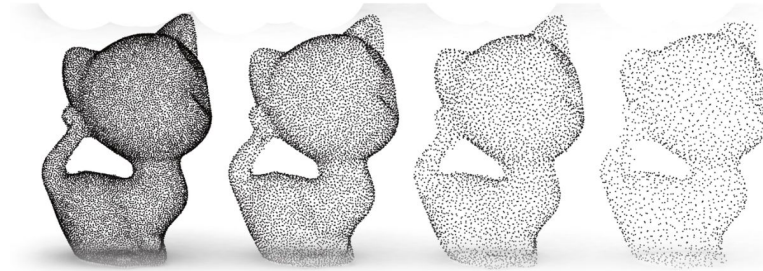
- Each voxel contains a binary value (indicating whether the voxel is on the surface or not),
- Most voxels are empty.

Point-cloud



- The simplest representation: only points, no connectivity.
- Collection of (x,y,z) coordinates, possibly with normal.
- Nearly all 3D scanning devices produce point clouds.

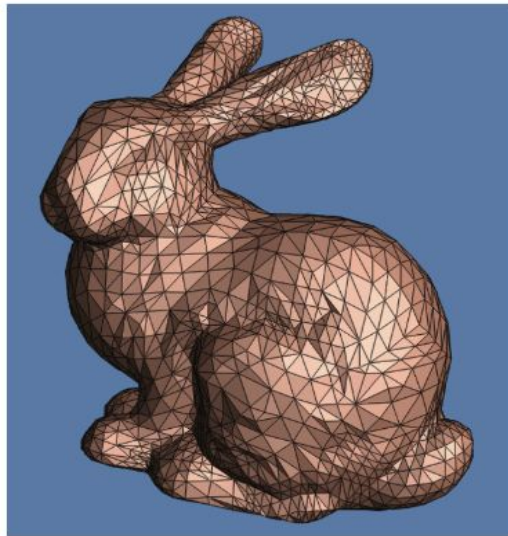
Point-cloud



- No surface/topology information;
- Weak approximation power; requires many points for the details.

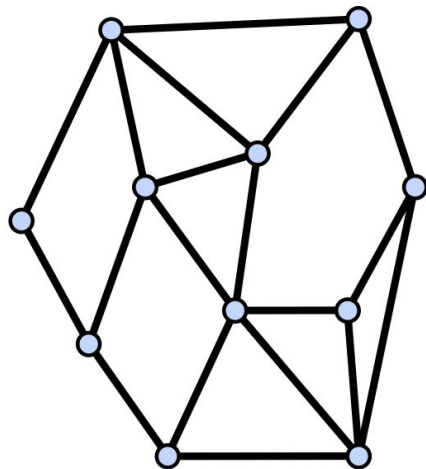
=> needs to be converted to the other representation for downstream applications.

Polygon Mesh



- The most popular representation for shapes in graphics.
- A compact form representing surfaces.
- A graph-like structure

Polygon Mesh

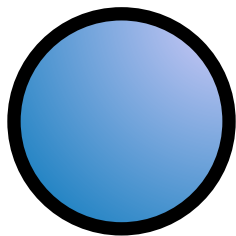


$$M = \langle V, E, F \rangle$$

vertices edges faces

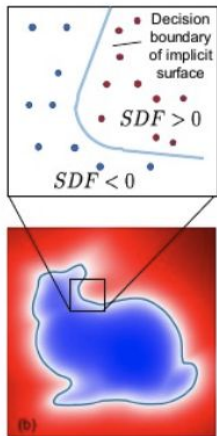
- A polygon mesh is a collection of vertices, edges and faces that defines the shape of a polyhedral object.
- A triangle mesh is a special case when all the faces are triangles.
- (+) Good for many applications:
 - Rendering
 - Texturing
 - Deformation / Manipulation
 - Simulation
- (-) Irregular structure

Implicit Representation



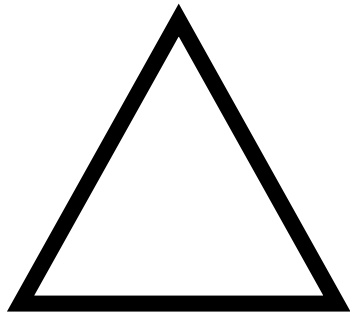
$$x^2 + y^2 + z^2 = 1$$

Implicit Representation

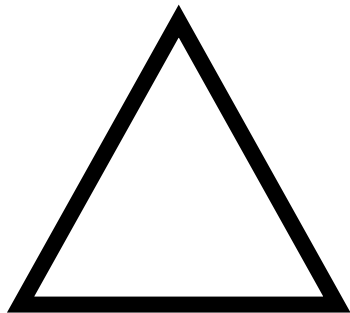


- A function that takes coordinates as input and returns occupancy or signed distance.
- Representation for output (generation) not input (processing)

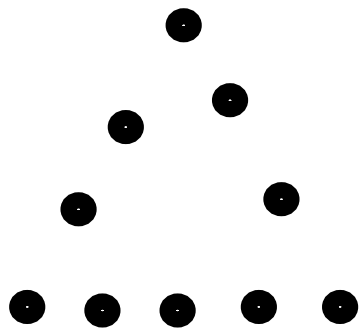
Park et al., DeepSDF, CVPR 2020.



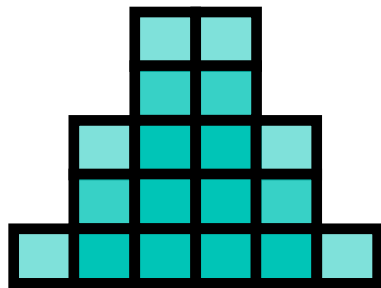
Target Geometry



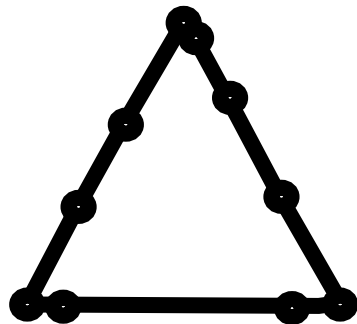
Target Geometry



point cloud



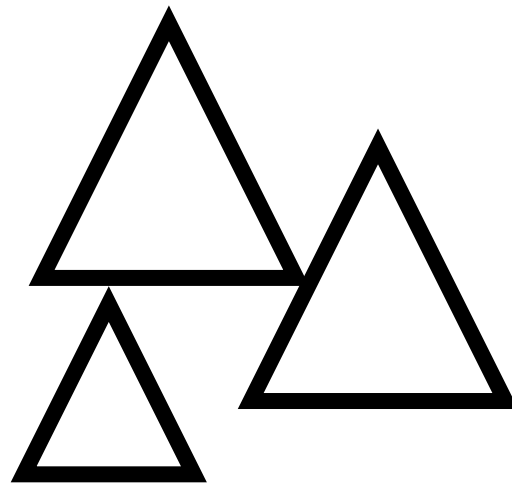
grid



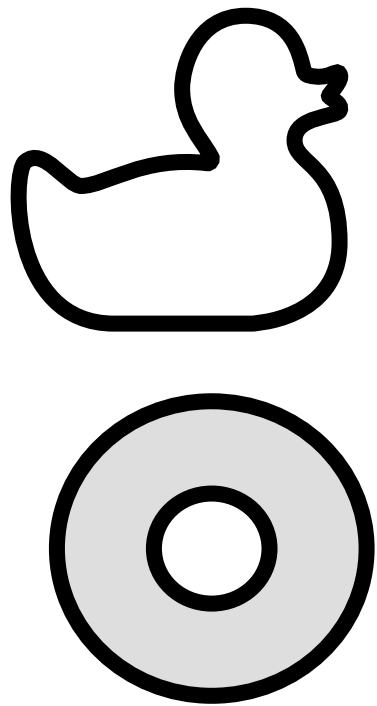
mesh

```
{ (x, y) |  
x ≥ 0,  
y ≥ 0,  
x + y ≤ 1 }
```

implicit

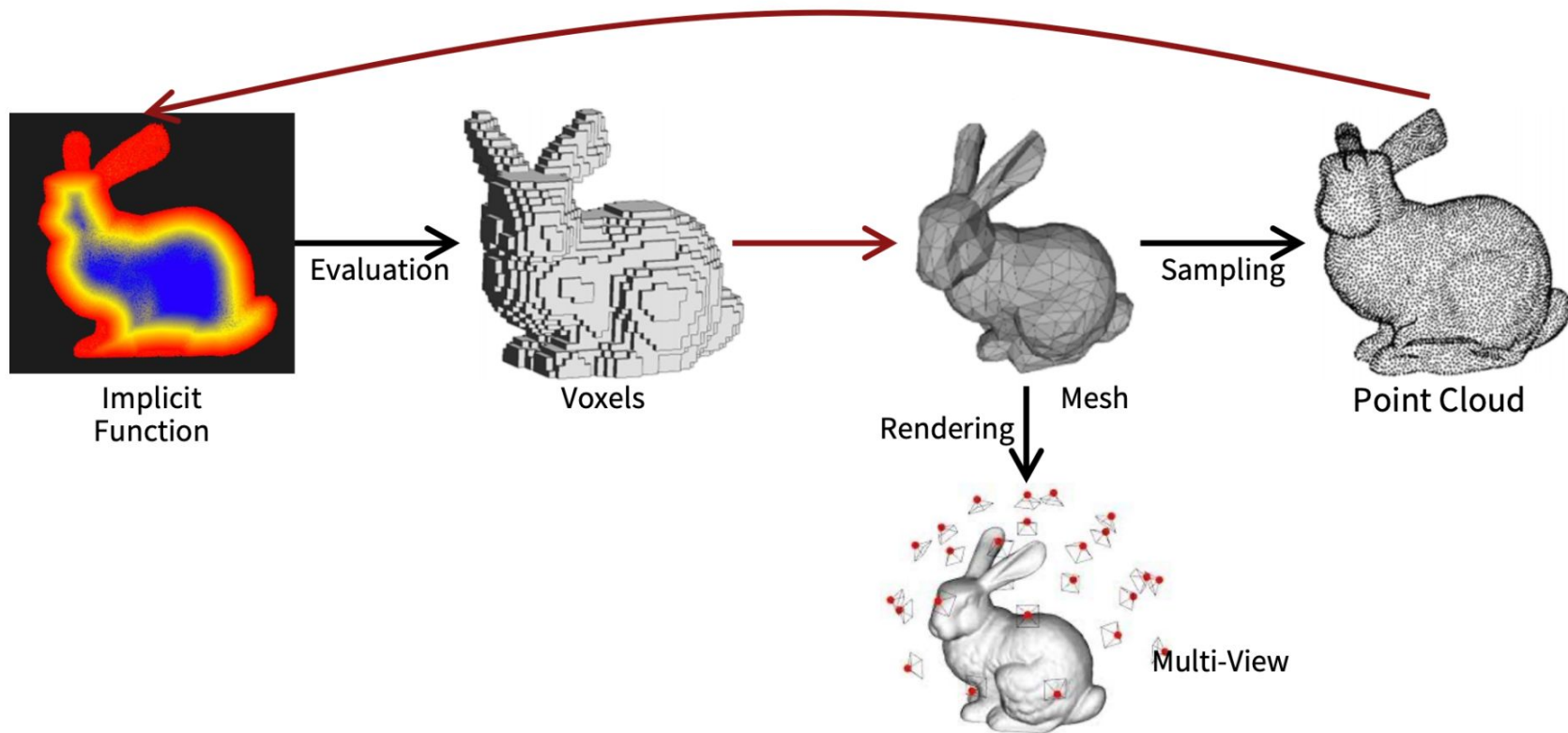


Target Geometry



Target Geometry

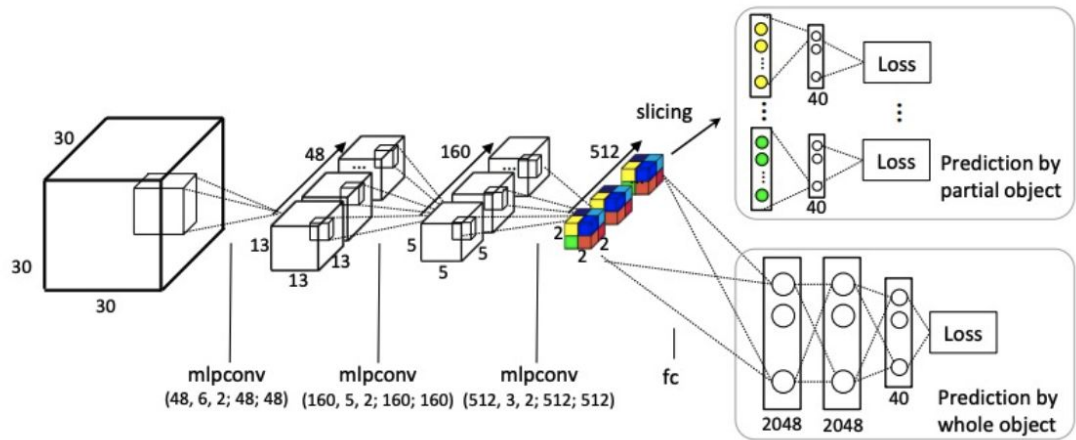
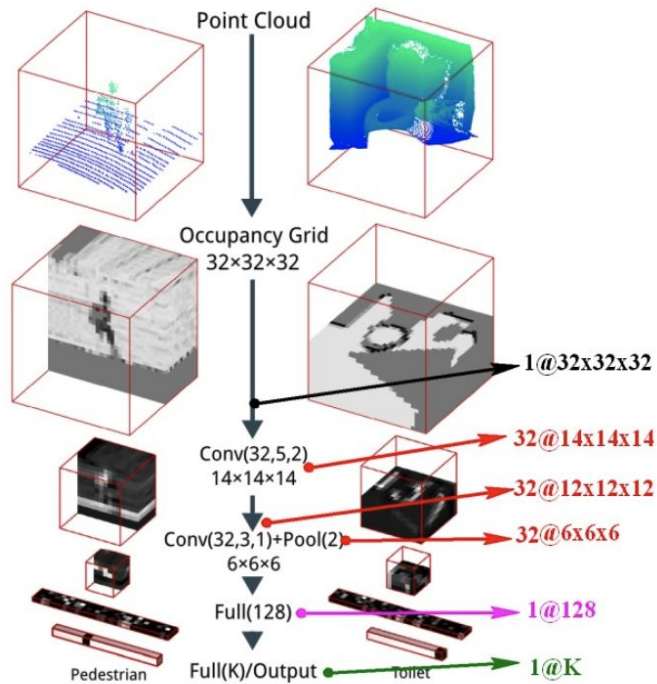
3D representations



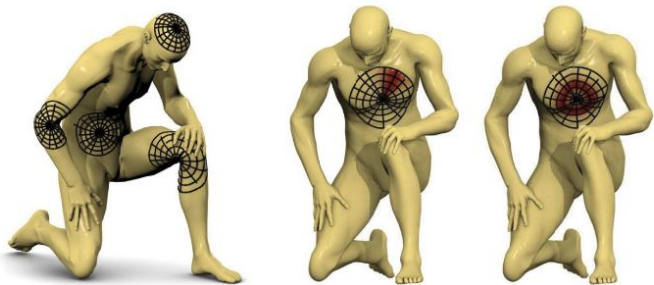
Content of today's lecture

- 3D / Geometric Data
 - Intro
 - 3D representations
- 3D Perception / Understanding
 - Processing
 - PointNet
- 3D Generation

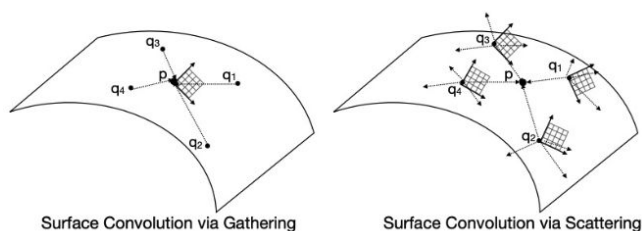
3D CNNs



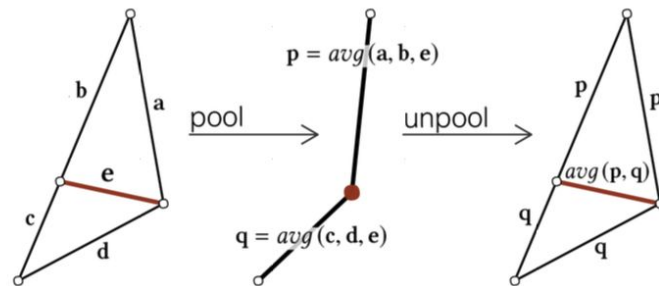
Neural Networks for Meshes



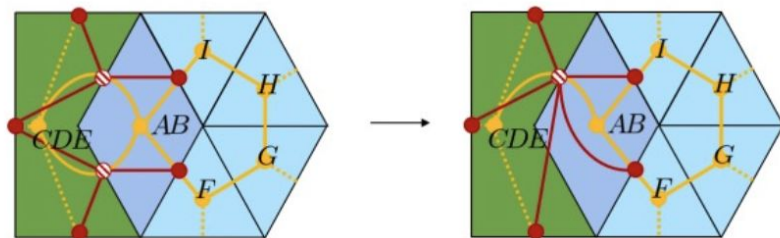
Masci et al., GCNN, ICCV 2015. v_q v_p



Mitchel et al., ICCV 2021.



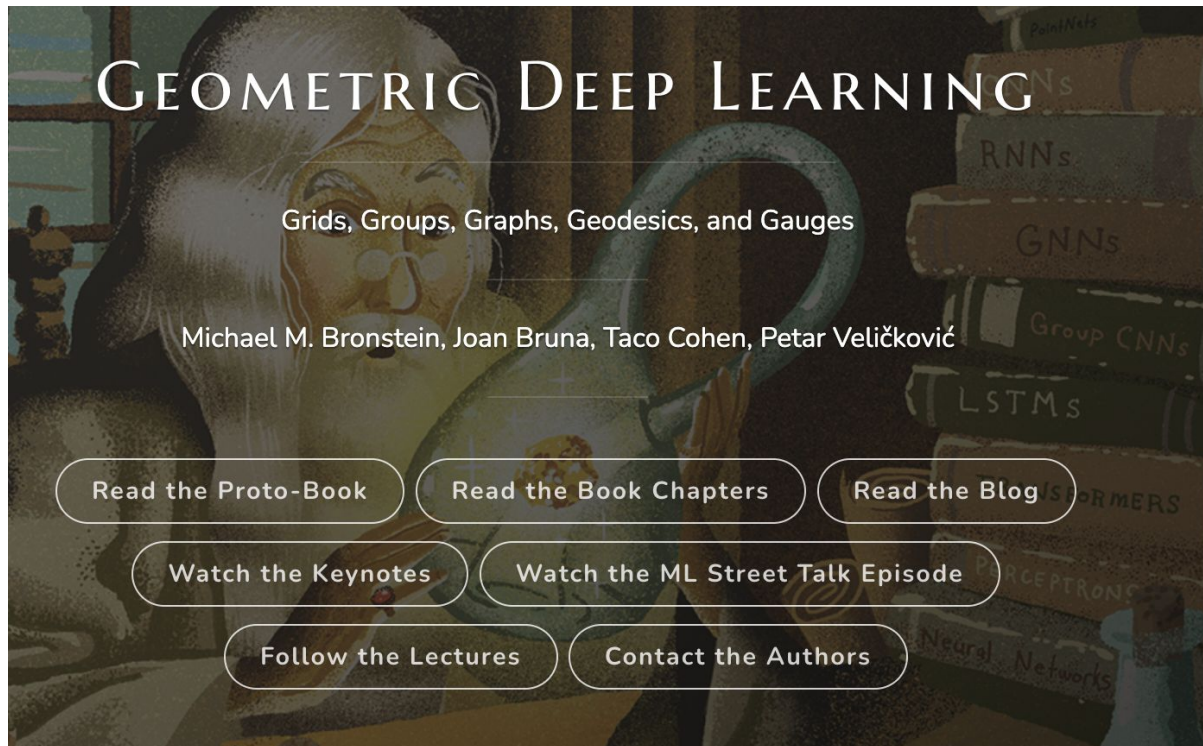
Hanocka et al., MeshCNN, SIGGRAPH 2019.



Milano et al., NeurIPS 2020.

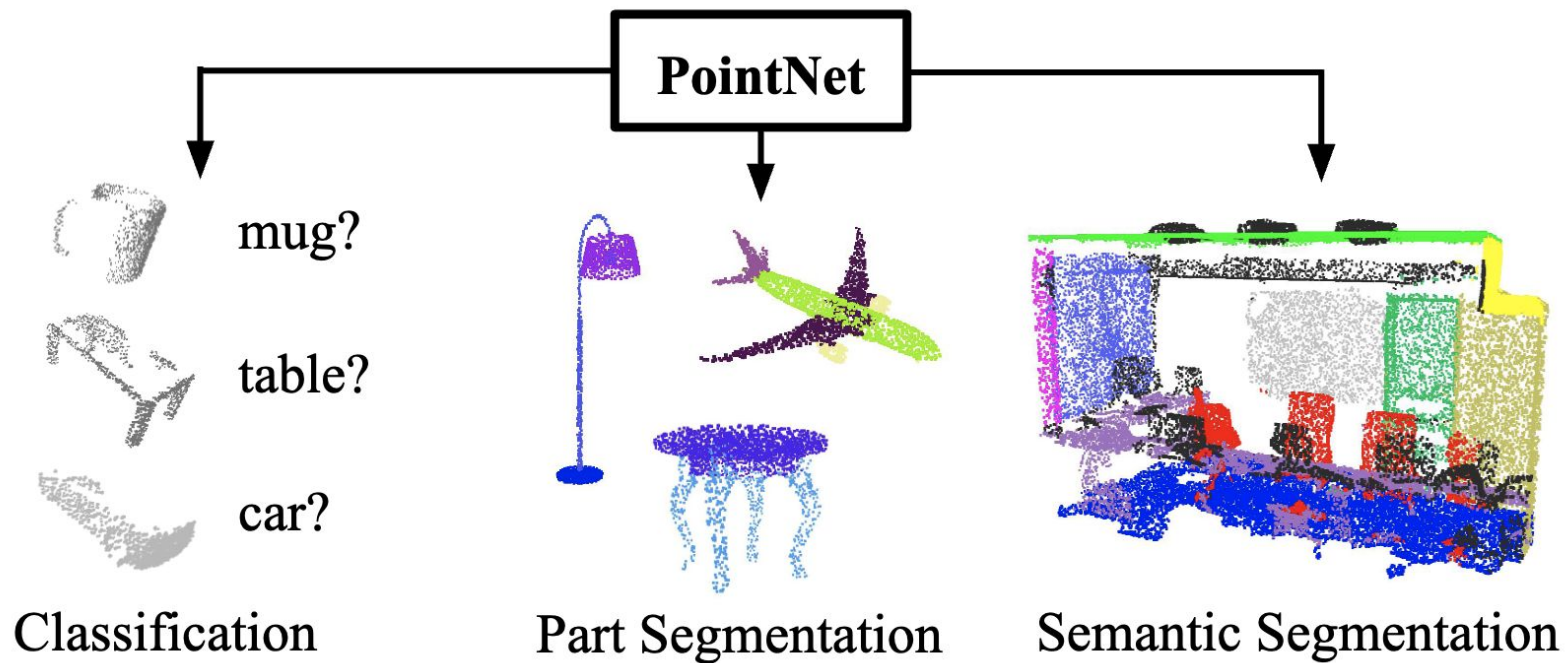
- Requires parameterization or specialized operations.
- Hard to implement. Verified only with a few use cases.

Neural Networks for Meshes



[Geometric Deep Learning](#)

PointNet



Input:

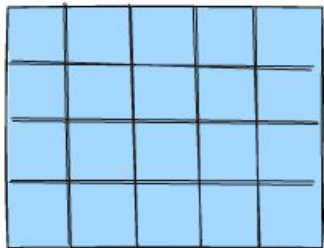
Point cloud:

N unordered points



D

N



Input:

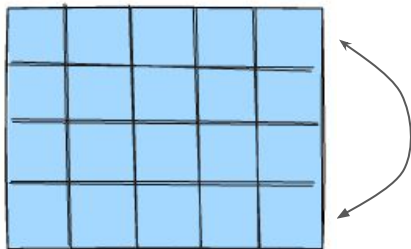
Point cloud:

N unordered points



D

N



• **Unordered.** Unlike pixel arrays in images or voxel arrays in volumetric grids, point cloud is a **set** of points without specific order.

=>

a network that consumes N 3D point sets needs to be invariant to $N!$ permutations of the input set.

Input:

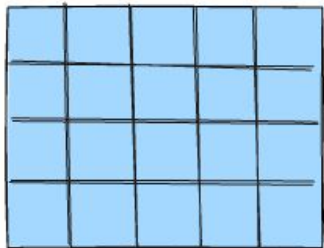
Point cloud:

N unordered points



D

N



• **Interaction among points.** The points are from a space with a distance metric. It means that points are not isolated, and neighboring points form a meaningful subset.

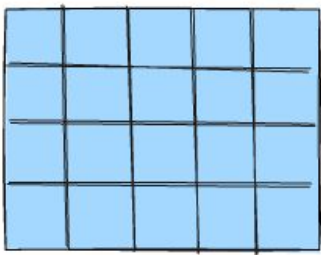
=>

the model needs to be able to capture local structures from nearby points, and the combinatorial interactions among local structures.

Input:

Point cloud:

N unordered points



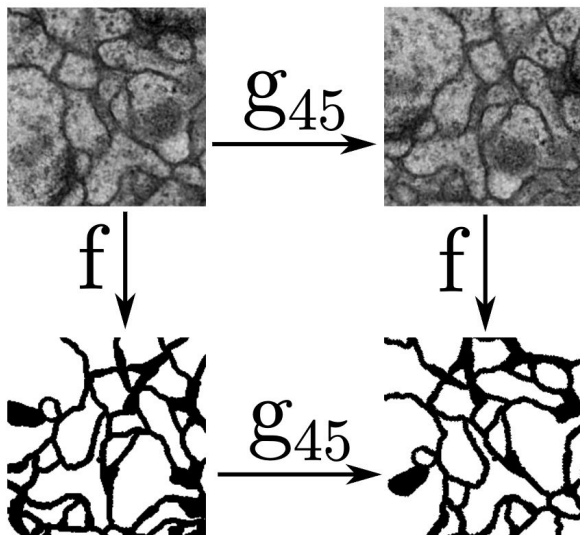
- ***Invariance under transformations.***

As a geometric object, the learned representation of the point set should be invariant to certain transformations.

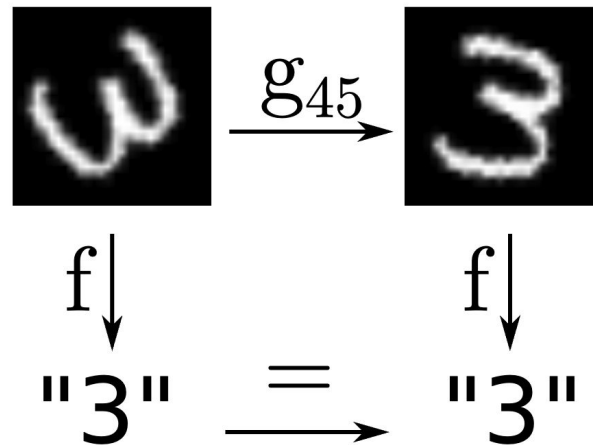
=>

rotating and translating points all together should not modify the global point cloud category nor the segmentation of the points.

Equivariant



Invariant



Permutation invariance

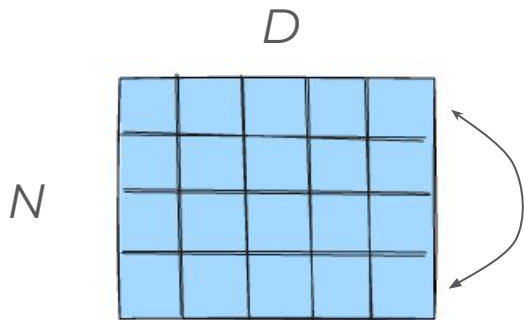
$$f(x_1, x_2, \dots, x_n) = y$$

$$f(x_{\pi_1}, x_{\pi_2}, \dots, x_{\pi_n}) = y$$

Permutation equivariance

$$f(x_1, x_2, \dots, x_n) = (y_1, y_2, \dots, y_n)$$

$$f(x_{\pi_1}, x_{\pi_2}, \dots, x_{\pi_n}) = (y_{\pi_1}, y_{\pi_2}, \dots, y_{\pi_n})$$



Symmetric Function

$$f(x_1, x_2, \dots, x_n) = f(x_{\pi_1}, x_{\pi_2}, \dots, x_{\pi_n}) \quad x_i \in \mathbb{R}$$

Symmetric Function

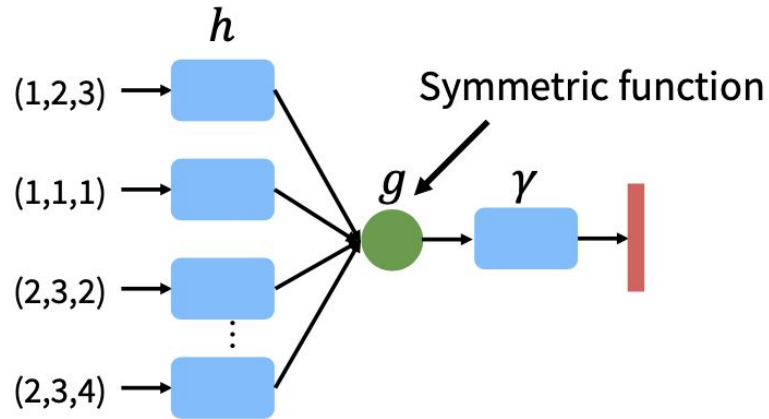
$$f(x_1, x_2, \dots, x_n) = f(x_{\pi_1}, x_{\pi_2}, \dots, x_{\pi_n}) \quad x_i \in \mathbb{R}$$

Examples:

$$f(x_1, x_2, \dots, x_n) = \max(x_1, x_2, \dots, x_n)$$

$$f(x_1, x_2, \dots, x_n) = x_1 + x_2 + \dots + x_n$$

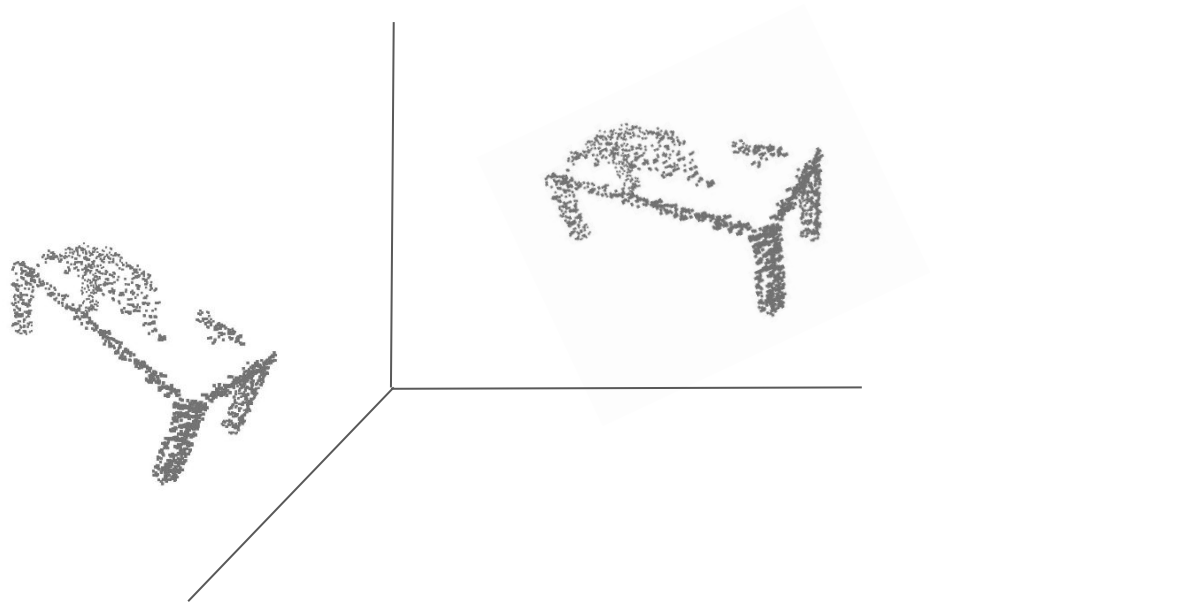
$$f(\{x_1, x_2, \dots, x_n\}) \approx g(h(x_1), h(x_2), \dots, h(x_n))$$



h Multi-Layer Perceptron(MLP)

g Max function(Max Pooling)

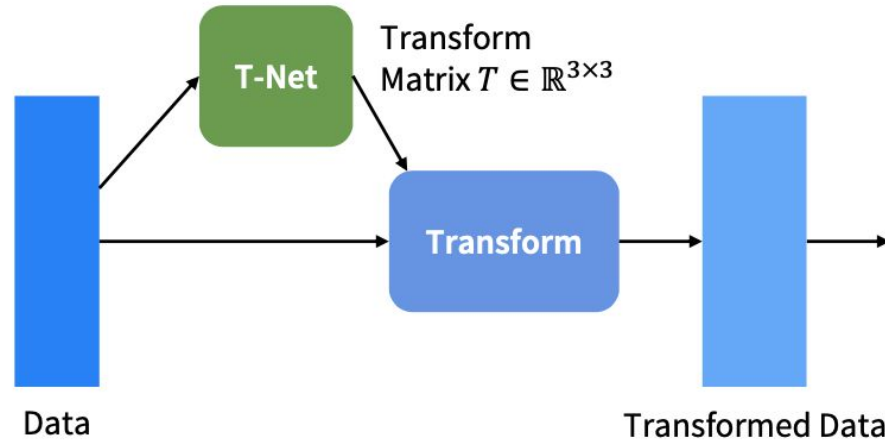
γ Multi-Layer Perceptron(MLP)



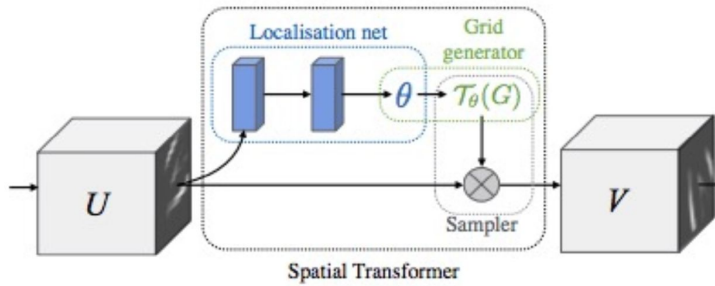
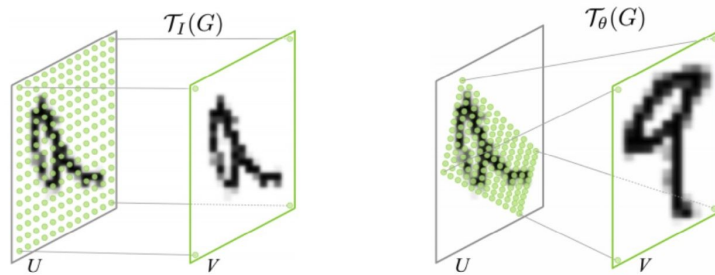
$$f(T \cdot x) = f(x)$$

f - Classifier

T - Transformation matrix



[\[1506.02025\] Spatial Transformer Networks](#)



batch = 0/200 theta = $\begin{bmatrix} 1.02 & 0.02 & -0.02 \\ -0.02 & 1.02 & -0.02 \end{bmatrix}$

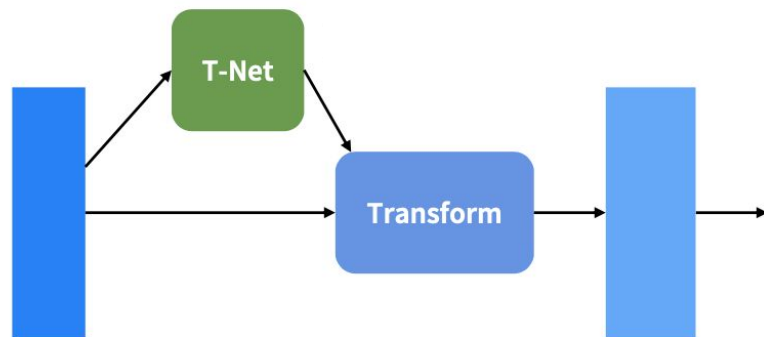


batch = 0/200 theta = $\begin{bmatrix} 0.98 & 0.02 & -0.02 \\ 0.02 & 1.02 & -0.02 \end{bmatrix}$



[\[1506.02025\] Spatial Transformer Networks](#)

Transform Matrix:
 3×3



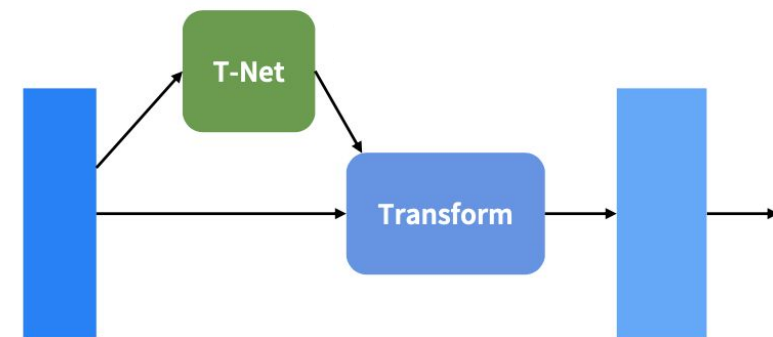
Data:
 $N \times 3$

Transformed Data:
 $N \times 3$

Regularization:
Transform matrix
 $K \times K$ close to
orthonormal:

$$L_{reg} = ||I - T \cdot T^T||_F^2$$

Transform Matrix:
 $K \times K$



Data:
 $N \times K$

Transformed Data:
 $N \times K$

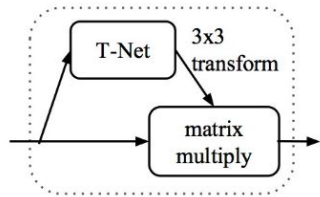
Regularization:
Transform matrix
 $K \times K$ close to
orthonormal:

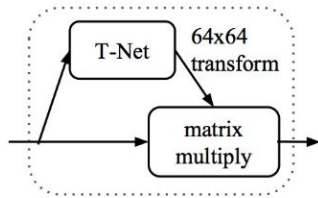
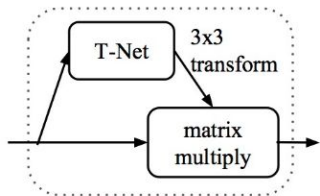
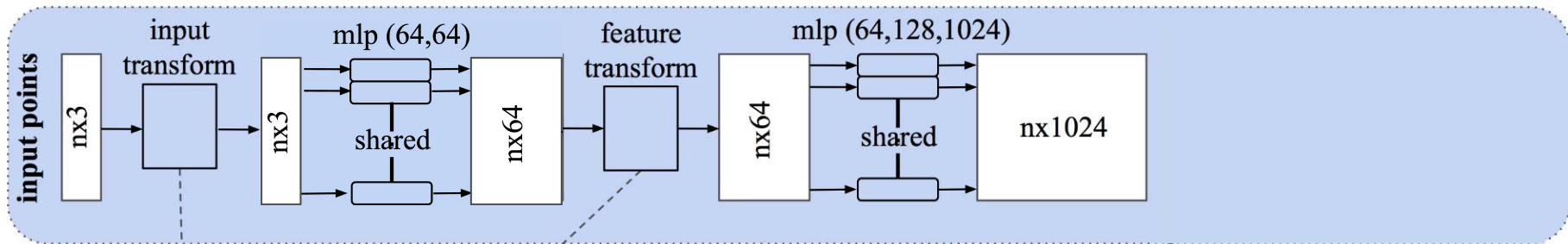
$$L_{reg} = ||I - T \cdot T^T||_F^2$$

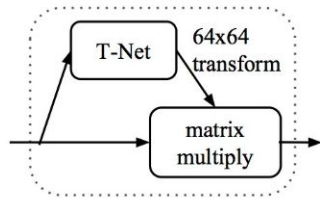
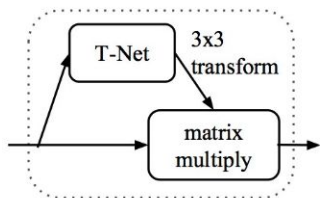
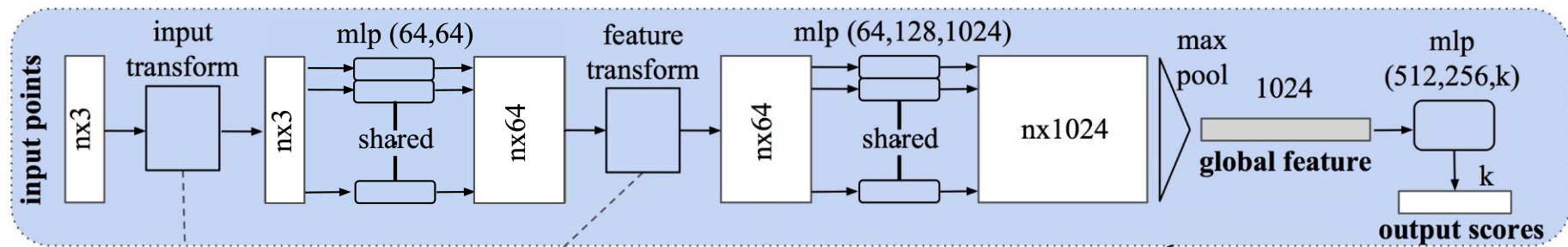
input points

$n \times 3$



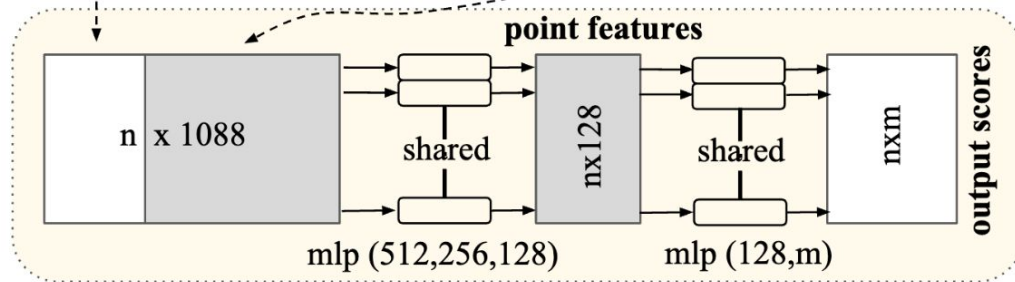
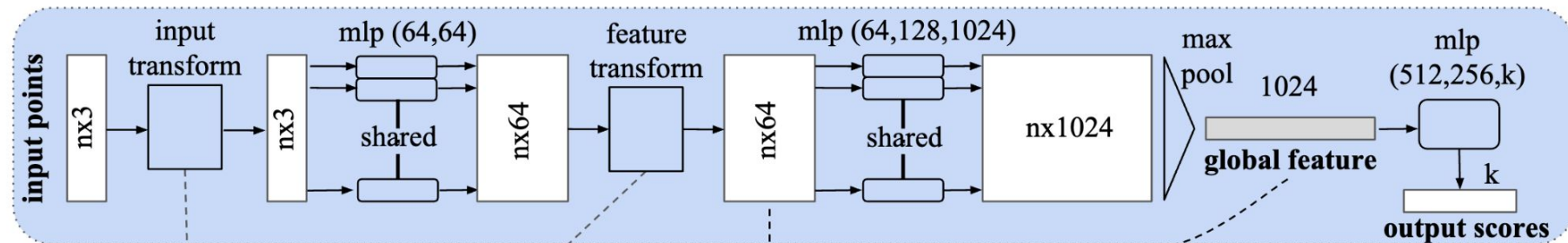






Extension to Segmentation Network

Classification Network



Segmentation Network

Classification – ModelNet Dataset

- 12,311 CAD Models
(9,843 for training /
2,468 for testing)
- 40 classes

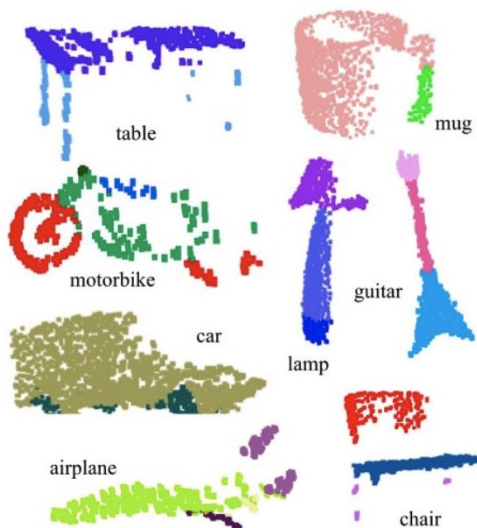


PRINCETON MODELNET

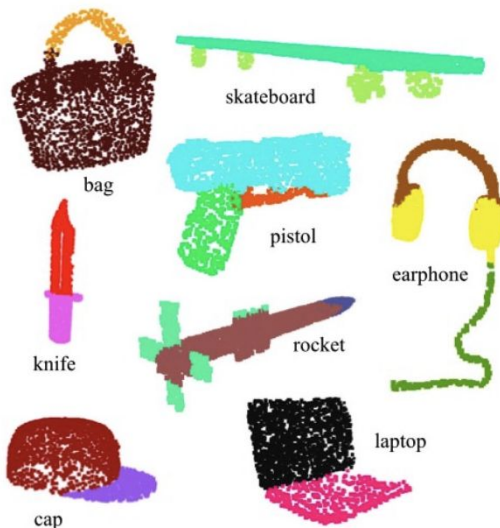


Wu et al., 3D ShapeNets: A Deep Representation for Volumetric Shapes, CVPR 2015.

Segmentation - ShapeNet Dataset



Partial Inputs



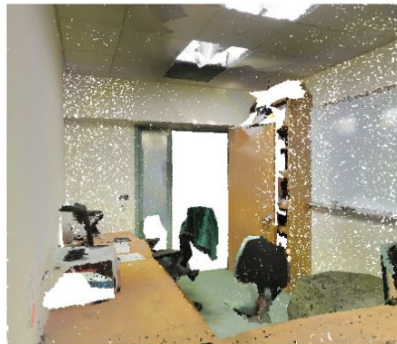
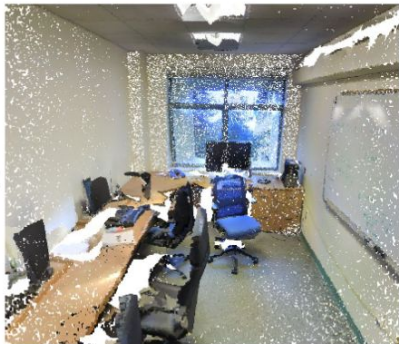
Complete Inputs

- 16,881 shapes.
- 16 classes.
- Each class has 2-5 part labels.

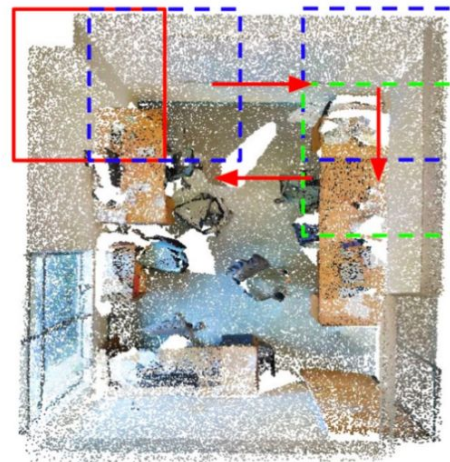
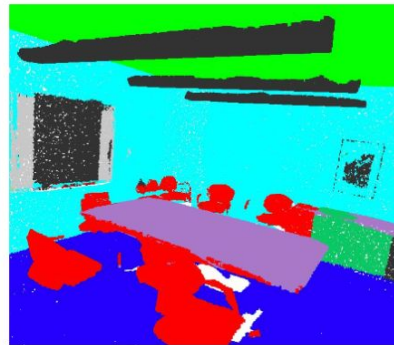
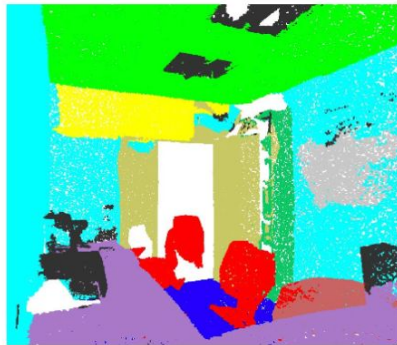
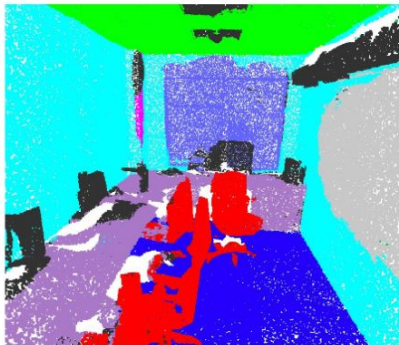


Segmentation - Stanford 2D-3D-S

Input



Output



	input	#views	accuracy avg. class	accuracy overall
SPH [11]	mesh	-	68.2	-
3DShapeNets [28]	volume	1	77.3	84.7
VoxNet [17]	volume	12	83.0	85.9
Subvolume [18]	volume	20	86.0	89.2
LFD [28]	image	10	75.5	-
MVCNN [23]	image	80	90.1	-
Ours baseline	point	-	72.6	77.4
Ours PointNet	point	1	86.2	89.2

ShapeNet (Part segmentation)

	mean	aero	bag	cap	car	chair	ear phone	guitar	knife	lamp	laptop	motor	mug	pistol	rocket	skate board	table
# shapes		2690	76	55	898	3758	69	787	392	1547	451	202	184	283	66	152	5271
Wu [27]	-	63.2	-	-	-	73.5	-	-	-	74.4	-	-	-	-	-	-	74.8
Yi [29]	81.4	81.0	78.4	77.7	75.7	87.6	61.9	92.0	85.4	82.5	95.7	70.6	91.9	85.9	53.1	69.8	75.3
3DCNN	79.4	75.1	72.8	73.3	70.0	87.2	63.5	88.4	79.6	74.4	93.9	58.7	91.8	76.4	51.2	65.3	77.1
Ours	83.7	83.4	78.7	82.5	74.9	89.6	73.0	91.5	85.9	80.8	95.3	65.2	93.0	81.2	57.9	72.8	80.6

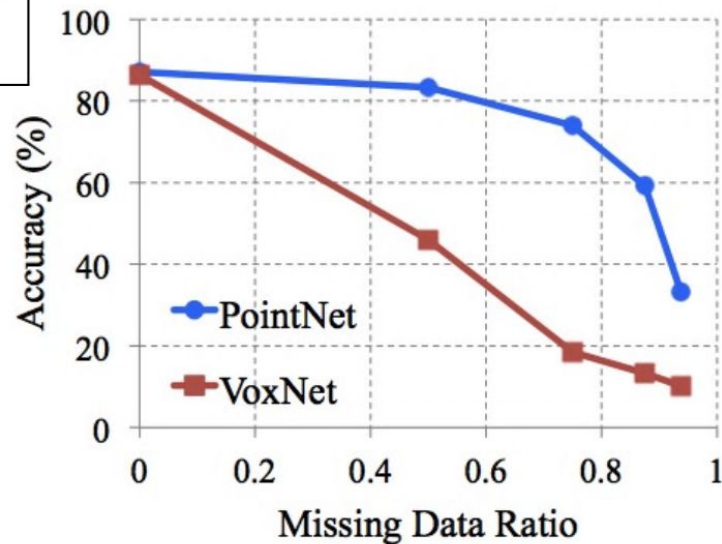
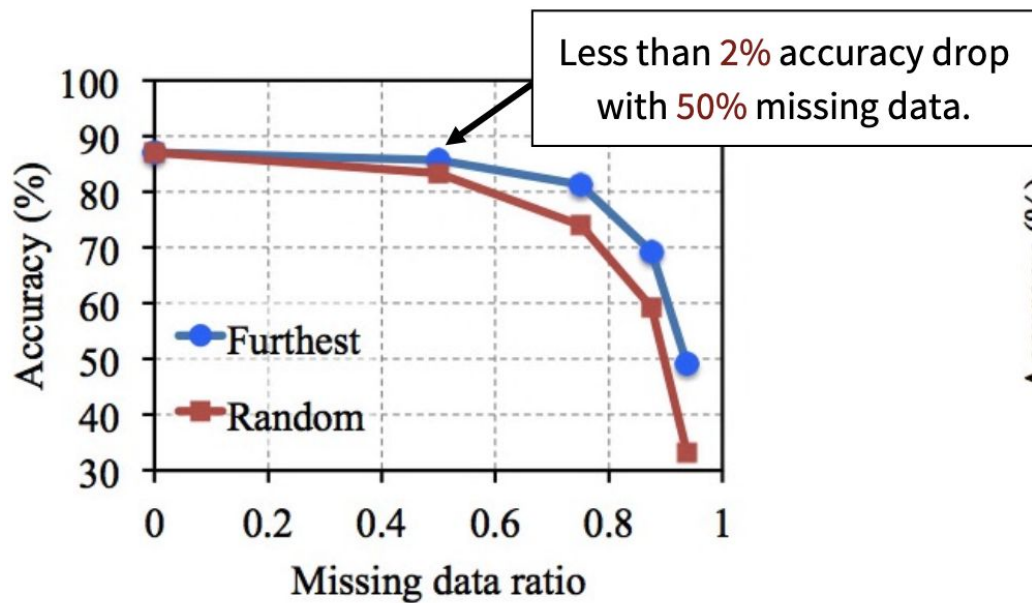
Stanford 2D-3D-S (Object segmentation)

	mean IoU	overall accuracy
Ours baseline	20.12	53.19
Ours PointNet	47.71	78.62

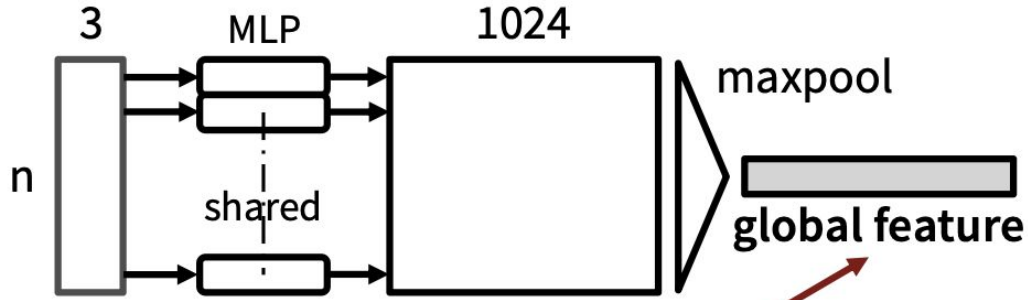
Efficiency of PointNet

	#params	FLOPs/sample
PointNet (vanilla)	0.8M	148M
PointNet	3.5M	440M
Subvolume [16]	16.6M	3633M
MVCNN [20]	60.0M	62057M

Robustness to Data Corruption



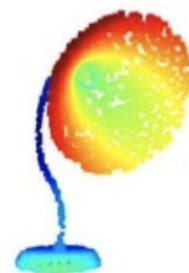
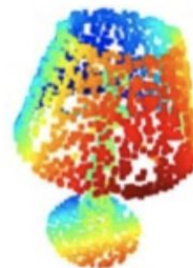
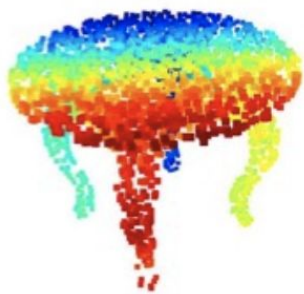
Visualizing Global Point Cloud Features



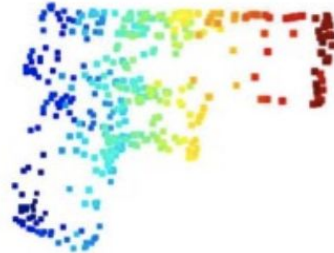
Which input points are contributing to the global feature?
(Critical points)

Visualizing Global Point Cloud Features

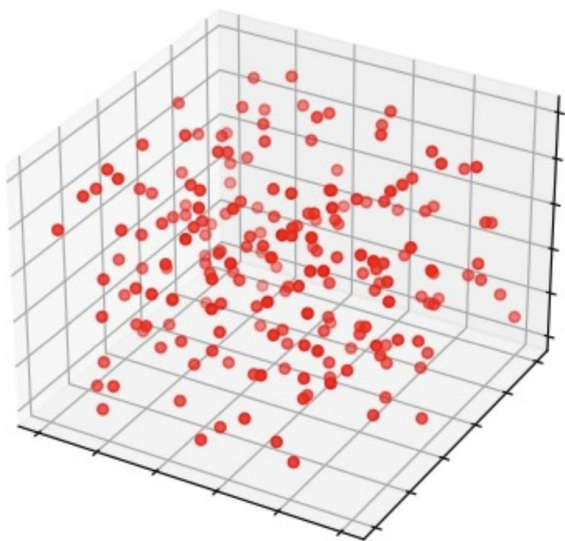
Original Shape:



Critical Point Sets:



Visualizing Global Point Cloud Features



Upper bound

Sample a point p in the 3D space and add the point if

$$h(p) < \max_{x_i \in S} h(x_i)$$

Find how many points can be added while not changing the global feature.

Visualizing Global Point Cloud Features

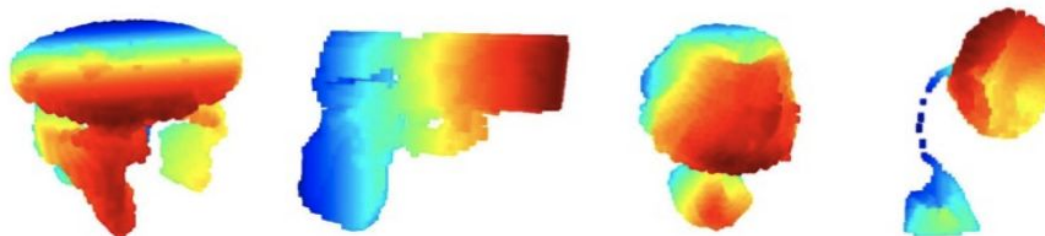
Original Shape:



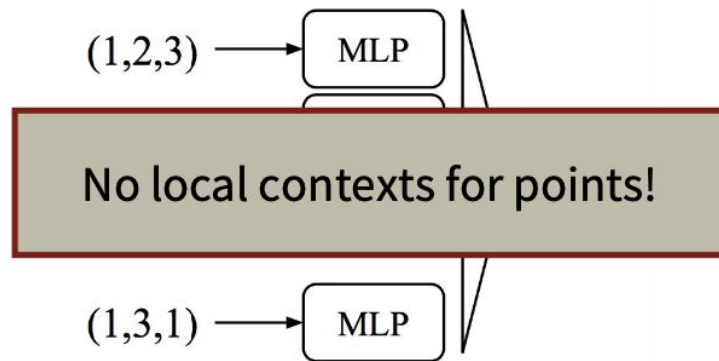
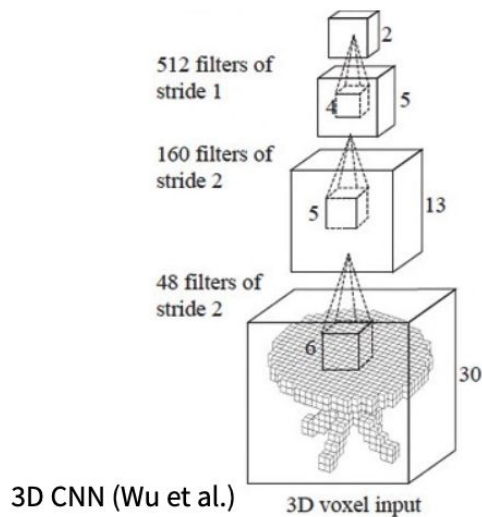
Critical Point Set:



Upper bound set:

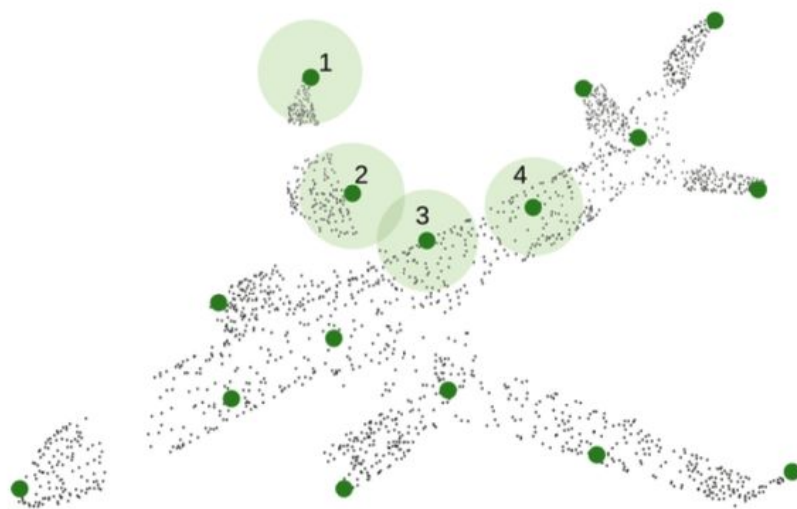
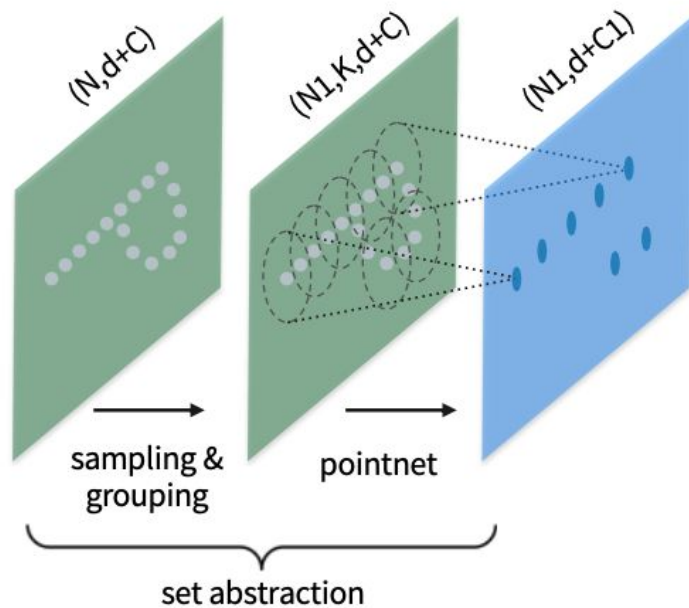


PointNet++



[\[1706.02413\] PointNet++: Deep Hierarchical Feature Learning on Point Sets in a Metric Space](#)
[\[2206.04670\] PointNeXt: Revisiting PointNet++ with Improved Training and Scaling Strategies](#)

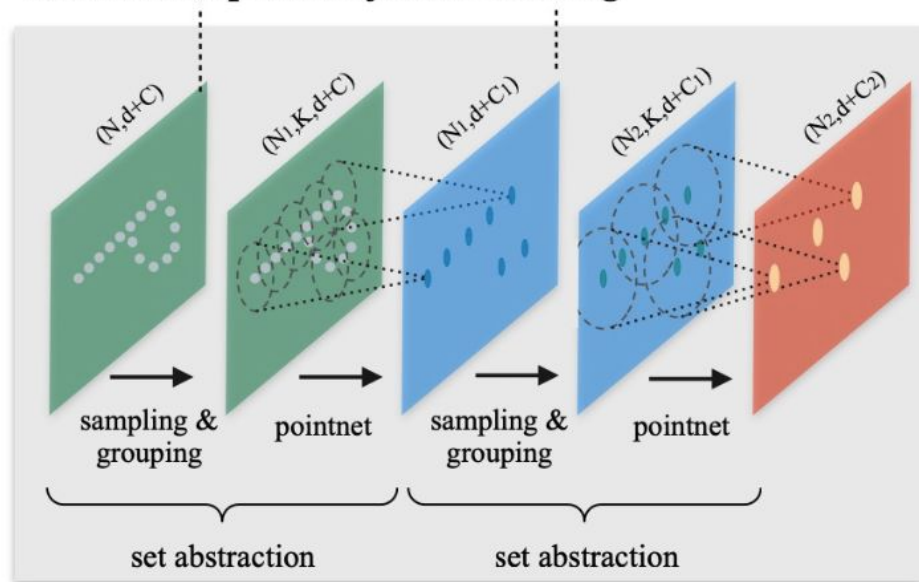
PointNet++



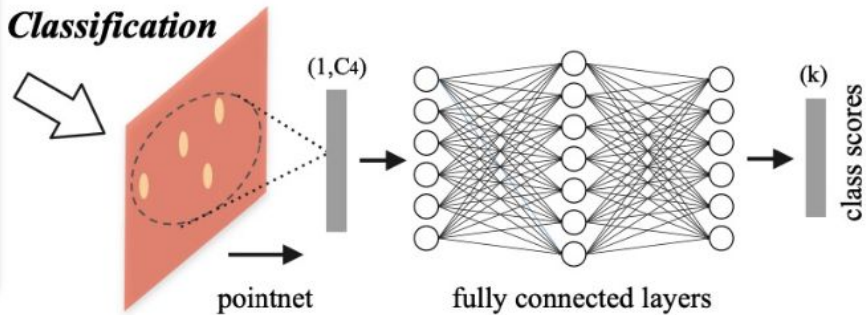
Xu et al., Grid-GCN, CVPR 2020.

PointNet++ for Classification

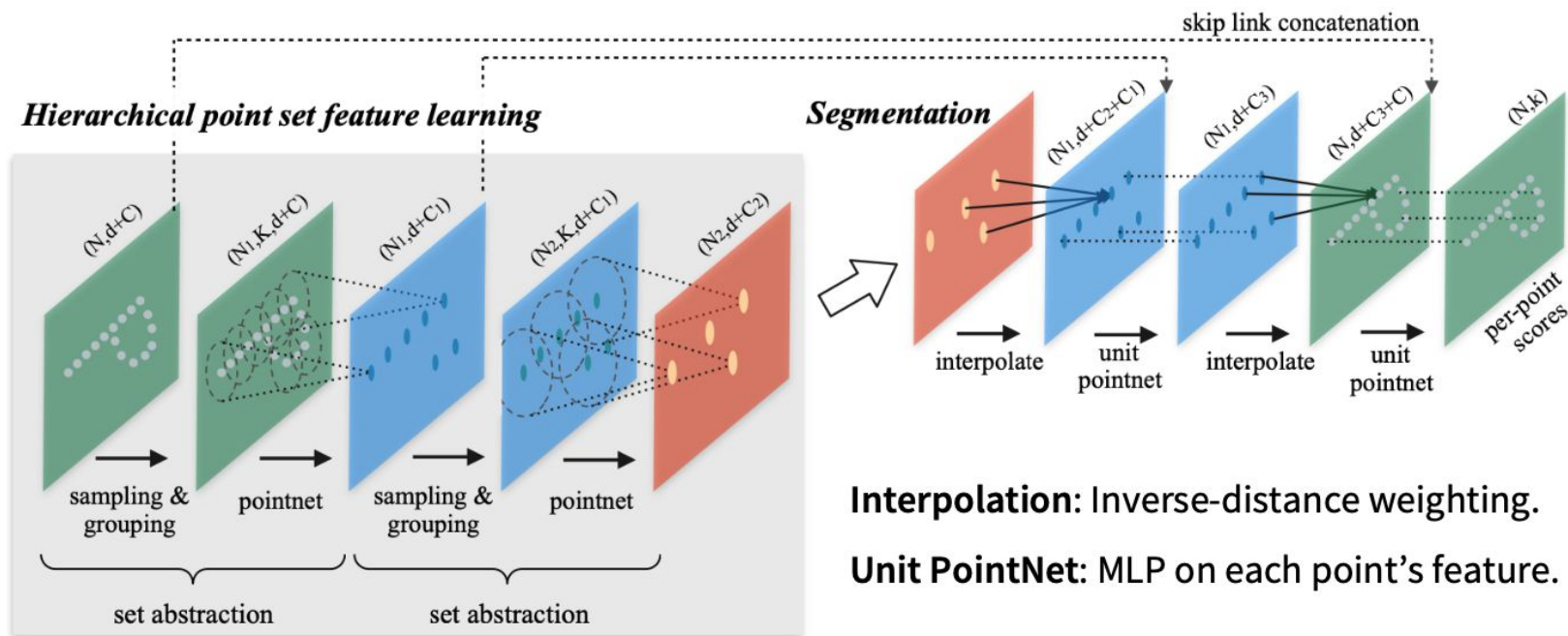
Hierarchical point set feature learning



Classification

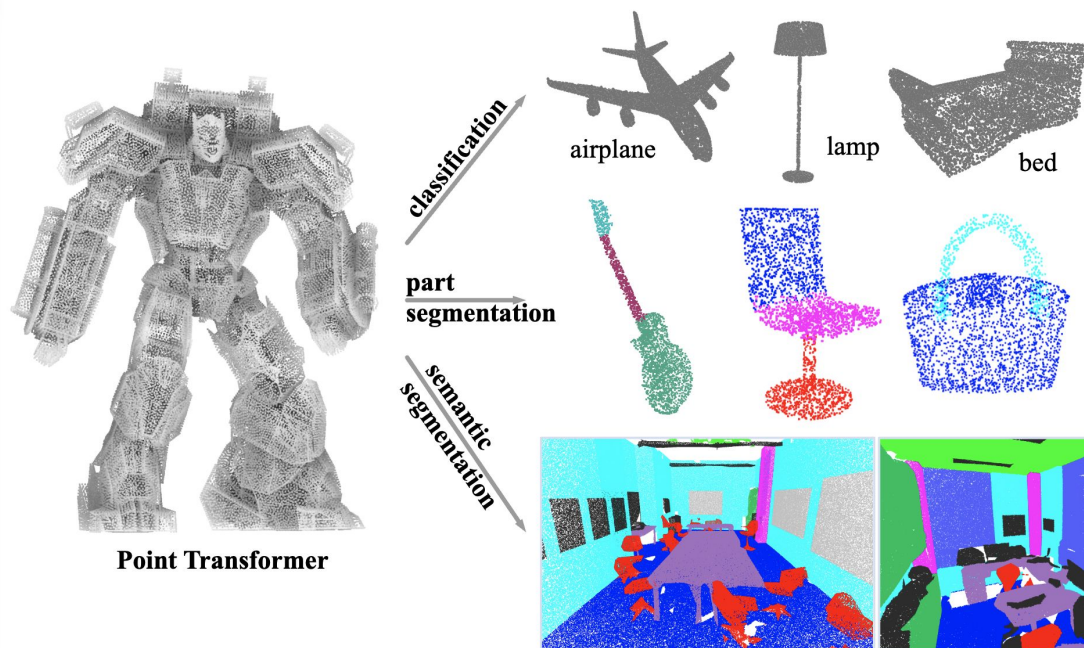


PointNet++ for Segmentation



Point Transformer

Point Transformer

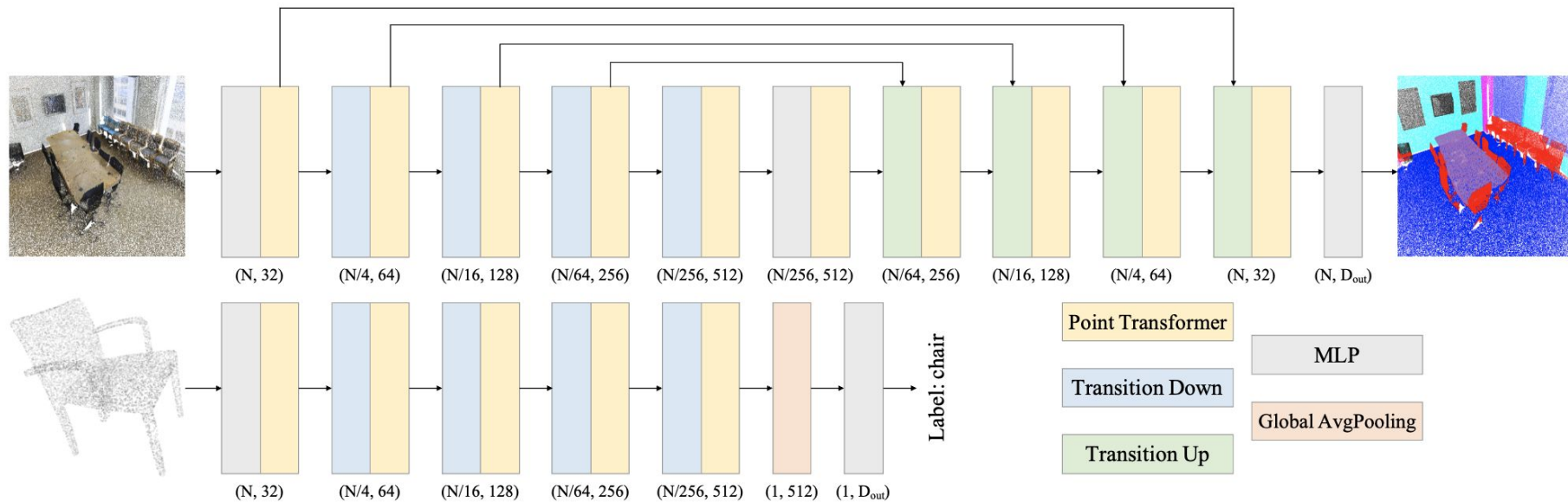


[\[2012.09164\] Point Transformer](#)

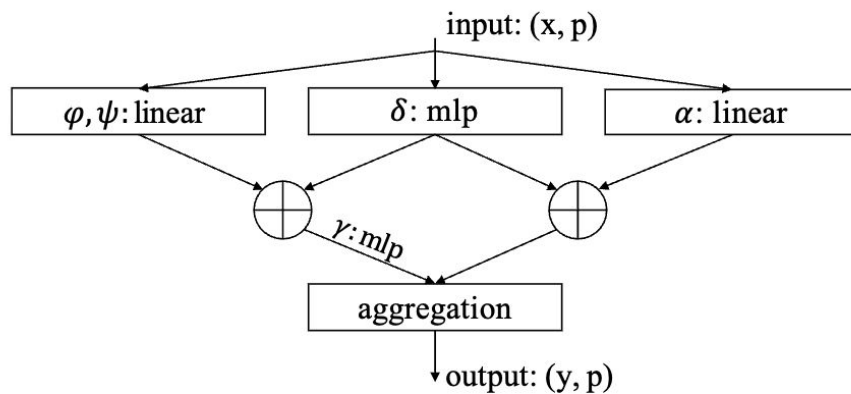
[\[2210.05666\] Point Transformer V2: Grouped Vector Attention and Partition-based Pooling](#)

[\[2312.10035\] Point Transformer V3: Simpler, Faster, Stronger](#)

Point Transformer



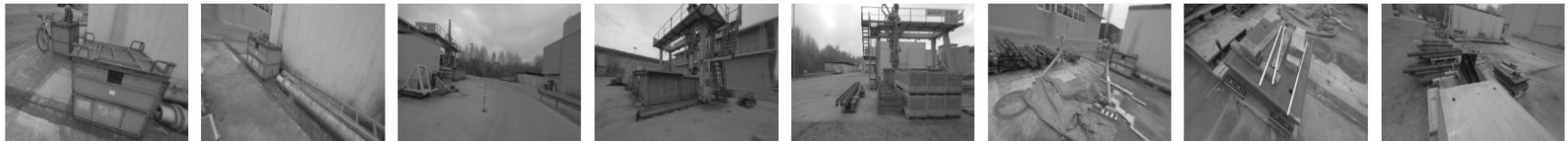
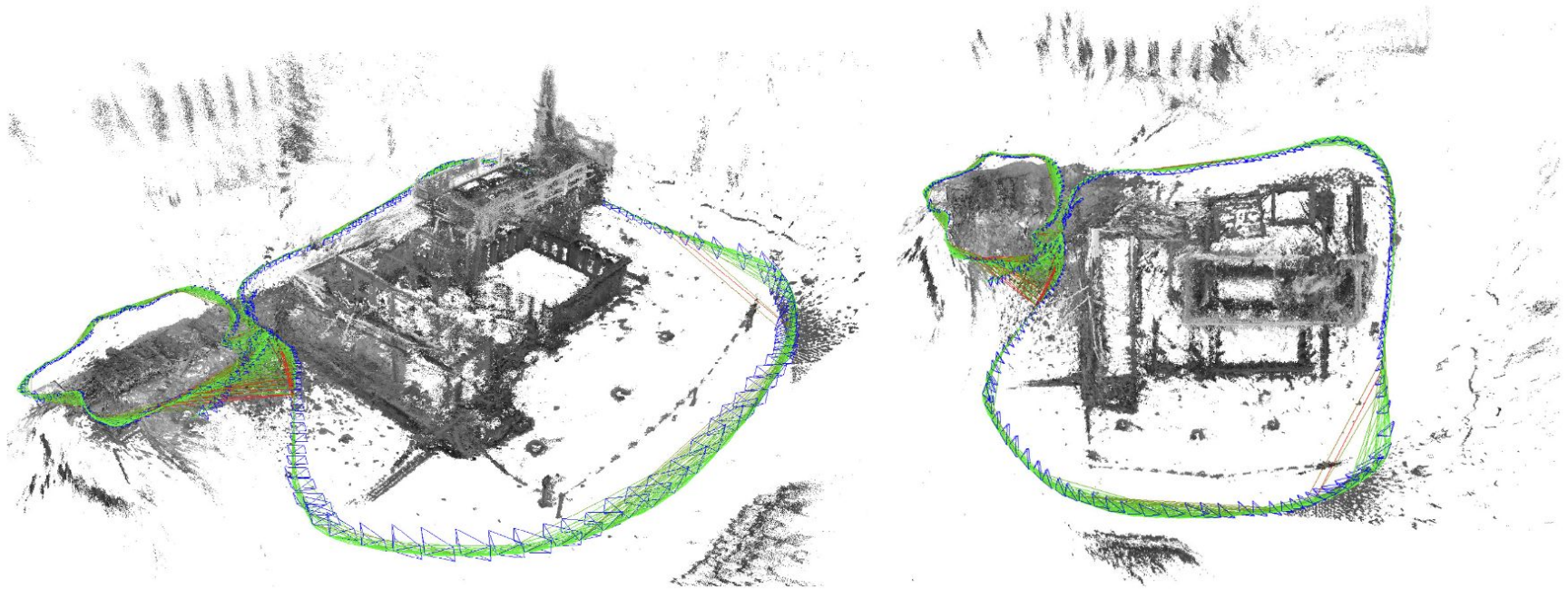
Point Transformer [Point transformer layer]



Content of today's lecture

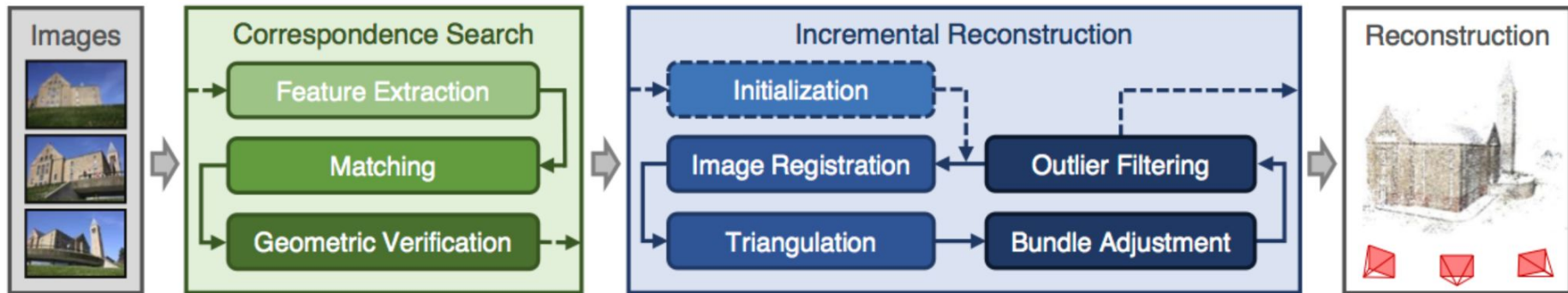
- 3D / Geometric Data
 - Intro
 - 3D representations
- 3D Perception / Understanding
 - Processing
 - PointNet
- 3D Generation
 - NeRF

3D perception (SLAM)



[LSD-SLAM: Large-scale direct monocular SLAM](#)

3D perception (SLAM)



- **Sensor Input:** Uses cameras (Visual SLAM), LiDAR, or other sensors to perceive the surroundings.
- **Feature Extraction:** Identifies key points and features in the visual data.
- **Pose Estimation:** Calculates the sensor's movement (translation & rotation) between frames.
- **Mapping:** Builds a consistent representation of the environment (e.g., point clouds, occupancy grids).
- **Loop Closure:** Recognizes previously visited places to correct accumulated errors (drift) and refine the map.

COLMAP

An off-the-shelf SfM (Structure from Motion) library.



Sparse model of central Rome using 21K photos produced by COLMAP's SfM pipeline.

<https://github.com/colmap/colmap>

SLAM demo

<https://github.com/Vincentqyw/image-matching-webui>

<https://huggingface.co/spaces/Realcat/image-matching-webui>

Neural Radiance Fields (NeRF)

[\[2003.08934\] NeRF: Representing Scenes as Neural Radiance Fields for View Synthesis](#)
[NeRF Tutorial ECCV 2022](#)

Problem Statement

Input:

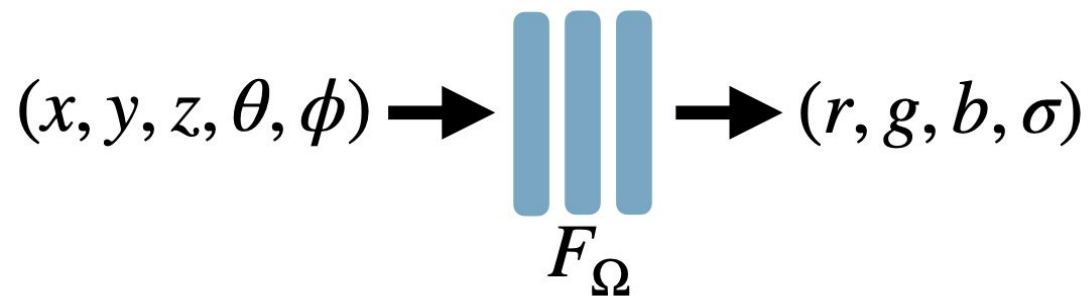
A set of calibrated Images

Output:

A 3D scene representation that renders novel views

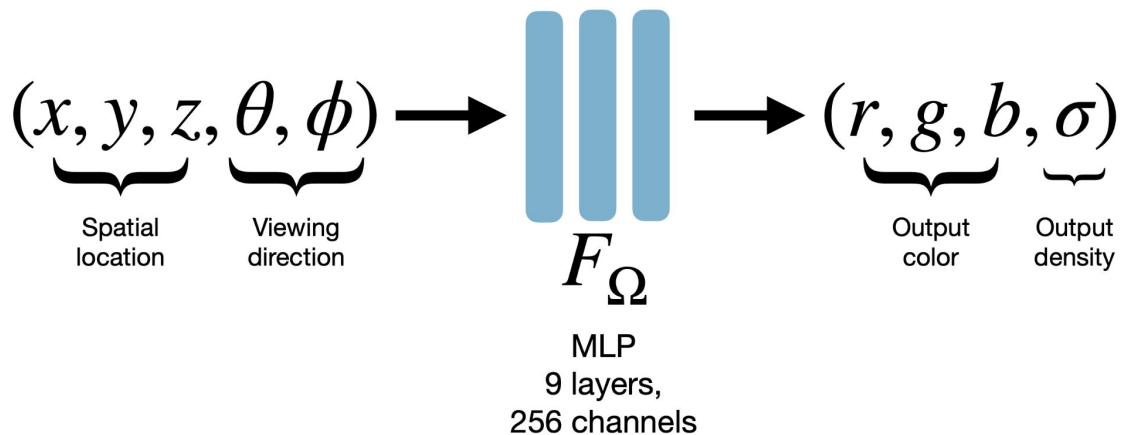


Neural Volumetric 3D Scene Representation



Neural Volumetric 3D Scene Representation

Representing a 3D scene as a continuous 5D function



Plenoptic Function

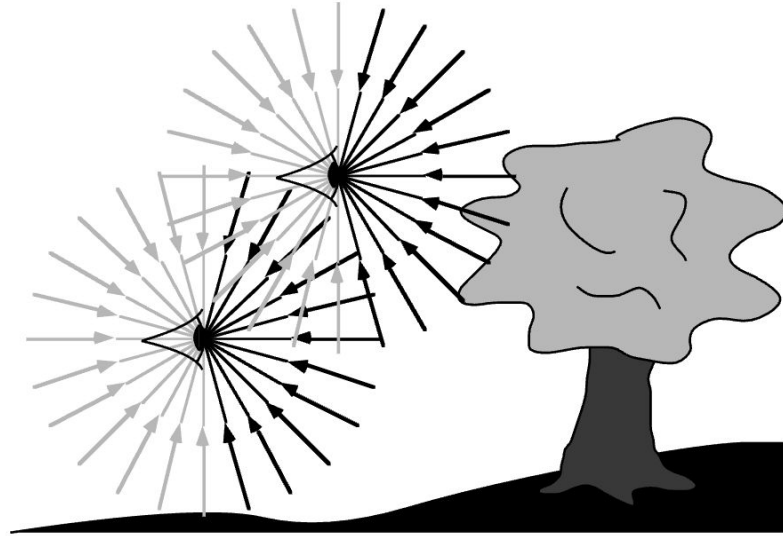
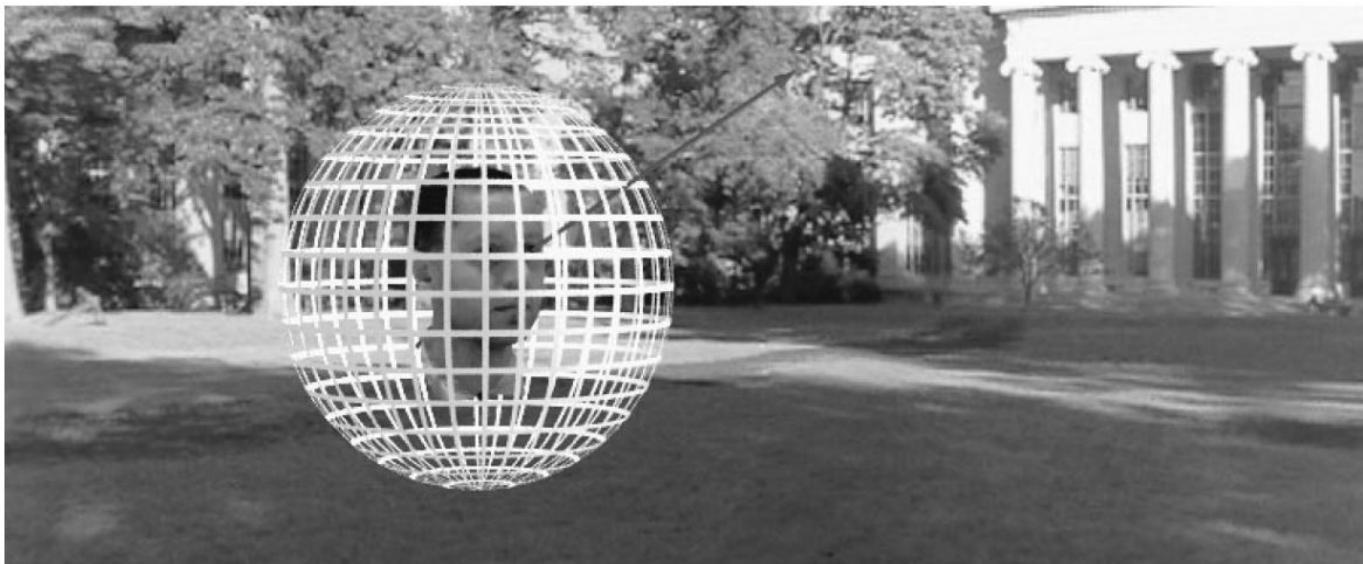


Fig.1.3

The plenoptic function describes the information available to an observer at any point in space and time. Shown here are two schematic eyes-which one should consider to have punctate pupils-gathering pencils of light rays. A real observer cannot see the light rays coming from behind, but the plenoptic function does include these rays.

[Adelson and Bergen. The plenoptic function and the elements of early vision. 1991.](#)

Plenoptic Function



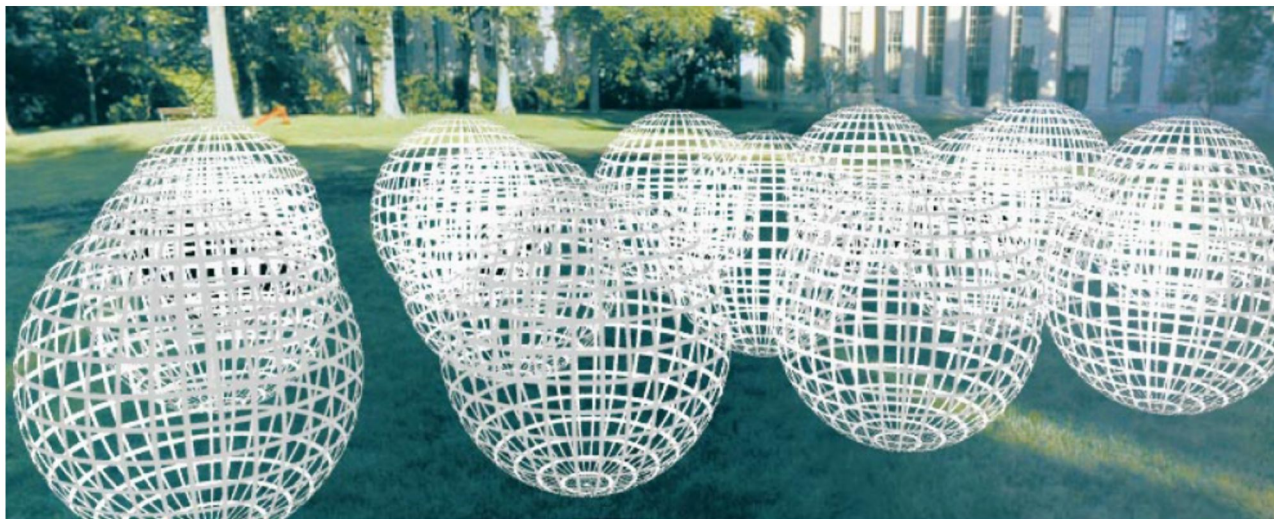
$$P(\theta, \phi)$$



$$P(\theta, \phi, \lambda, t)$$

Intensity of light

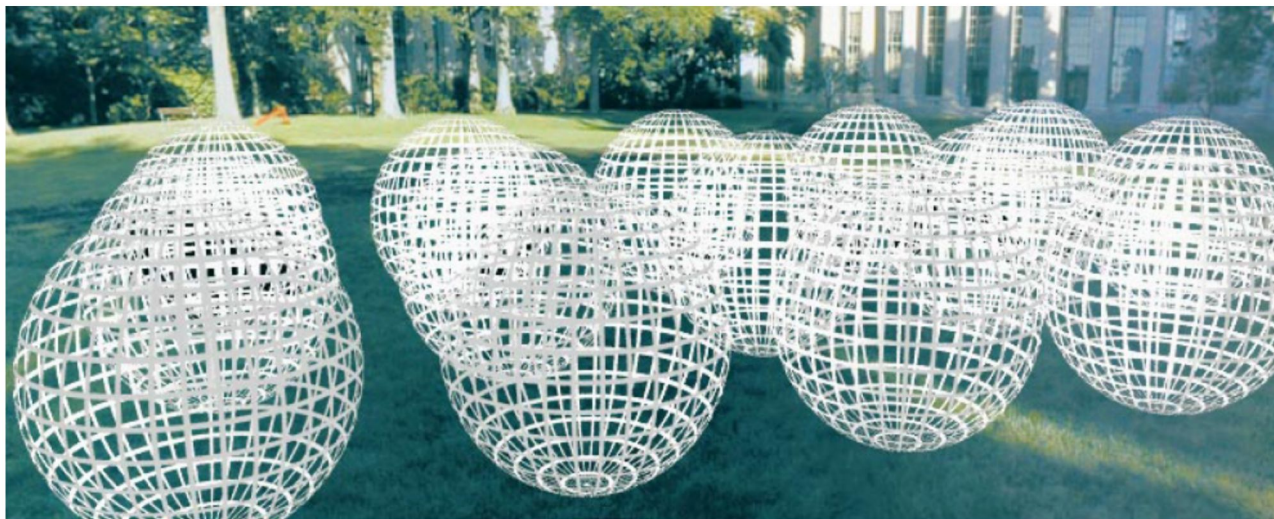
- Seen from a single position (viewpoint)
- Over time
- As a function of wavelength



$$P(\theta, \phi, \lambda, t, V_x, V_y, V_z)$$

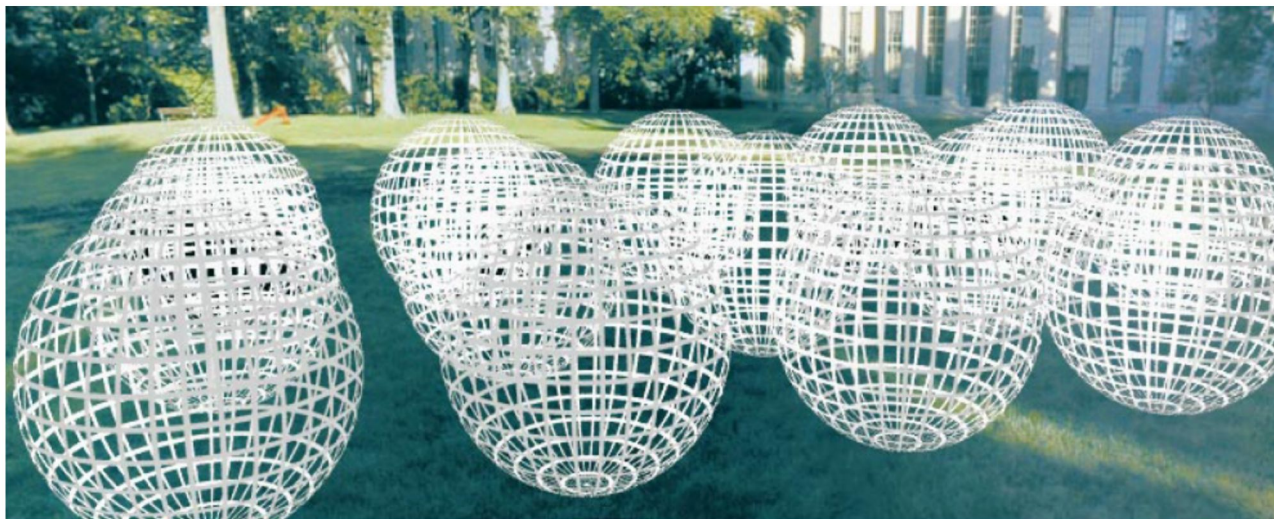
Intensity of light

- Seen from ANY position and direction
- Over time
- As a function of wavelength



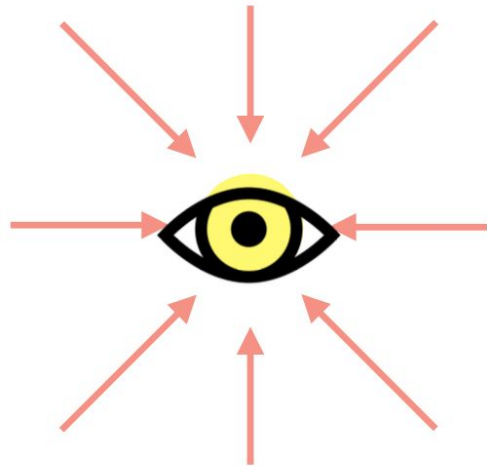
$$P(\theta, \phi, \lambda, t, V_x, V_y, V_z)$$

7D function, that can reconstruct every position & direction,
at every moment, at every wavelength
= it recreates the entirety of our visual reality!

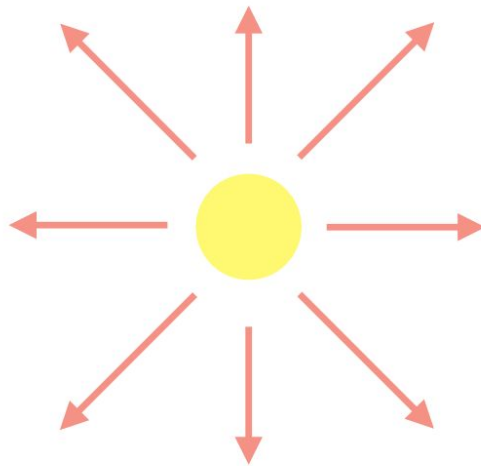


$$P(\theta, \phi, \cancel{\lambda}, \cancel{t}, V_x, V_y, V_z) \longrightarrow P(\theta, \phi, V_x, V_y, V_z)$$

1. Remove the time
2. Remove the wavelength & let the function output RGB colors

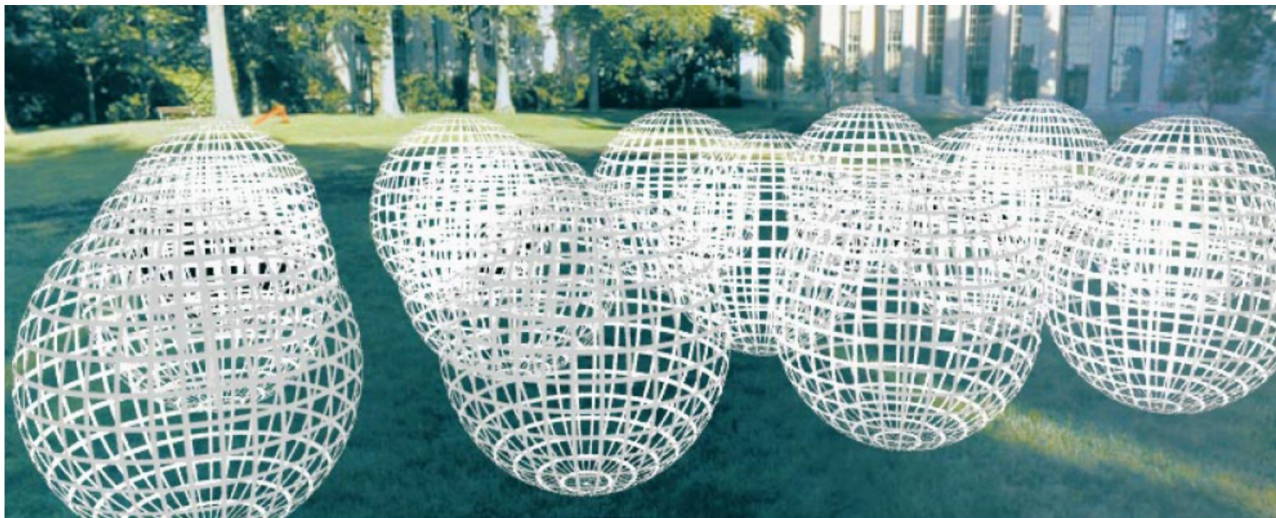


Plenoptic Function



NeRF



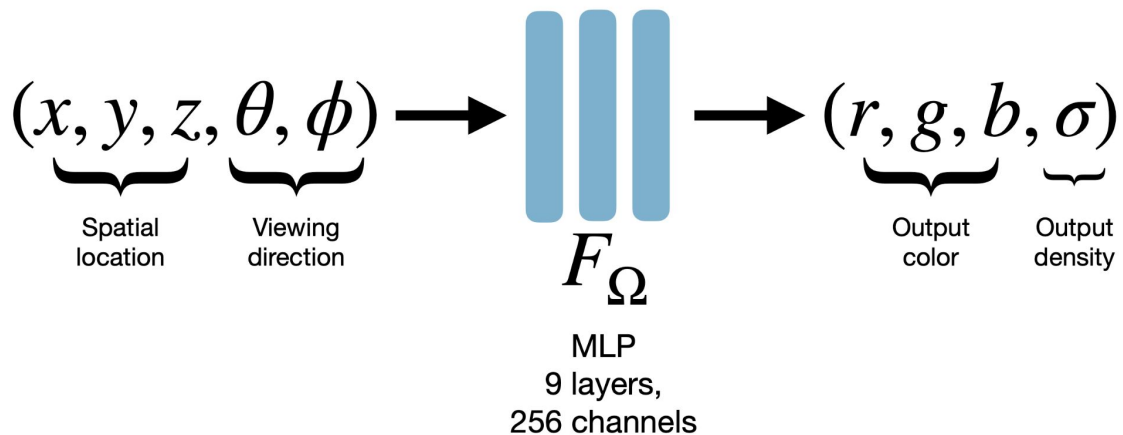


$$P(\theta, \phi, V_x, V_y, V_z)$$

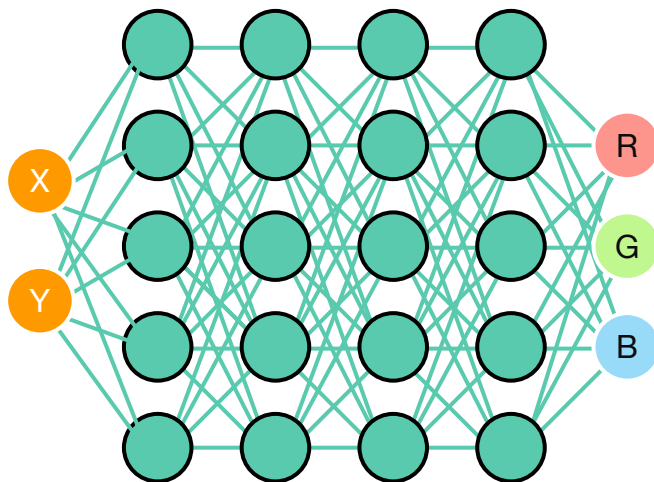
- For every location (3D), all possible views (2D)
- NeRF models this space with a continuous view-dependent volume with opacity
- The color emitted by every point is composited to render a pixel

Neural Volumetric 3D Scene Representation

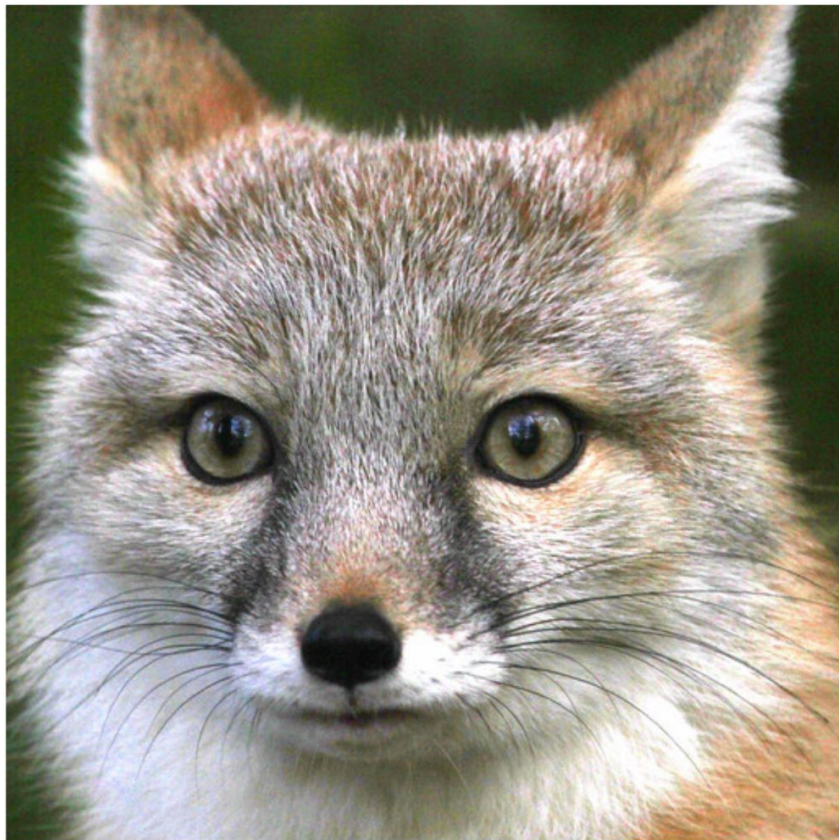
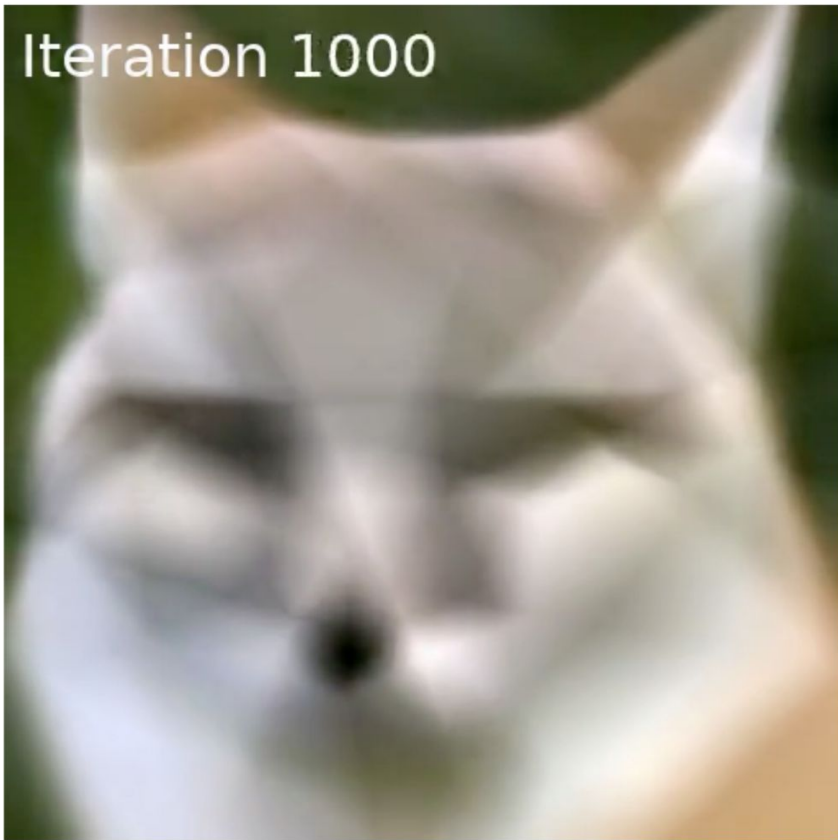
Representing a 3D scene as a continuous 5D function



Coordinate Based Neural Network



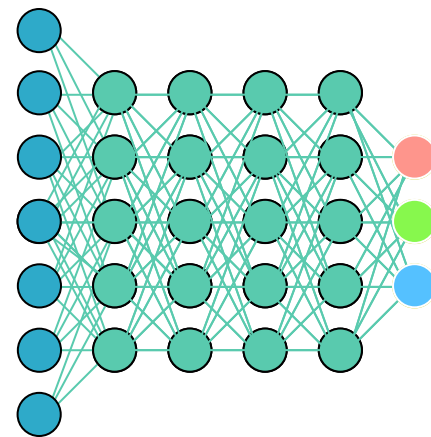
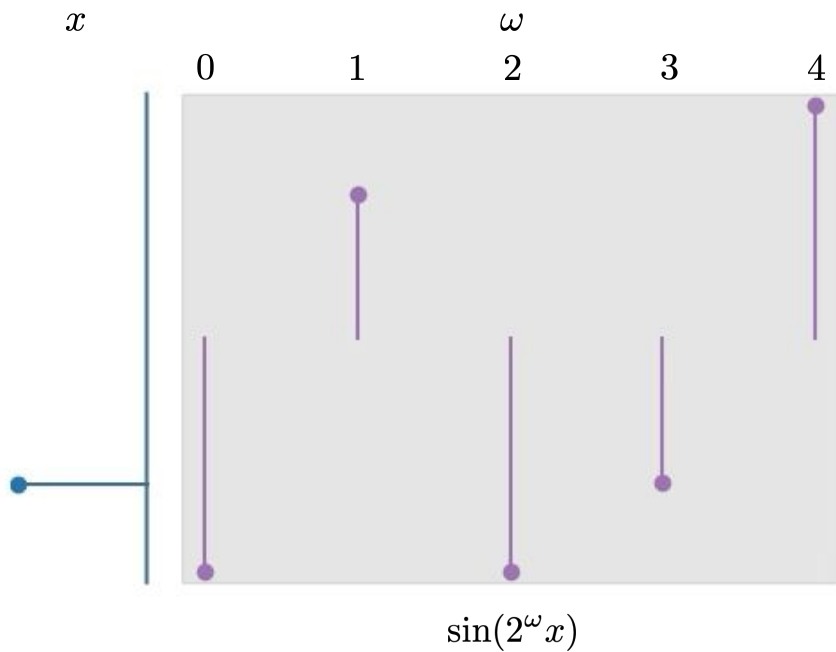
Iteration 1000



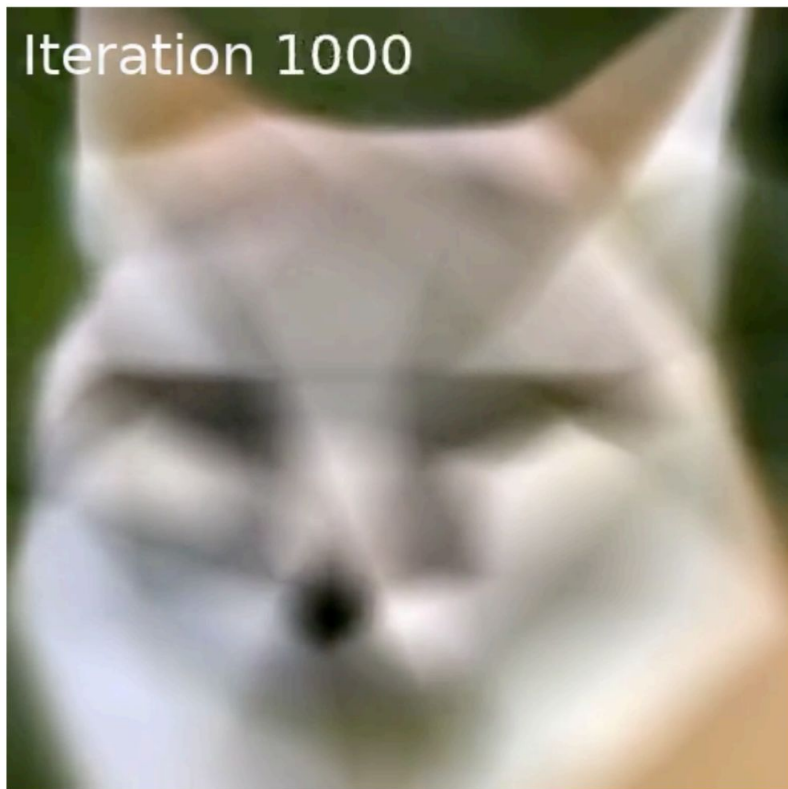
Positional Encoding

Standard input

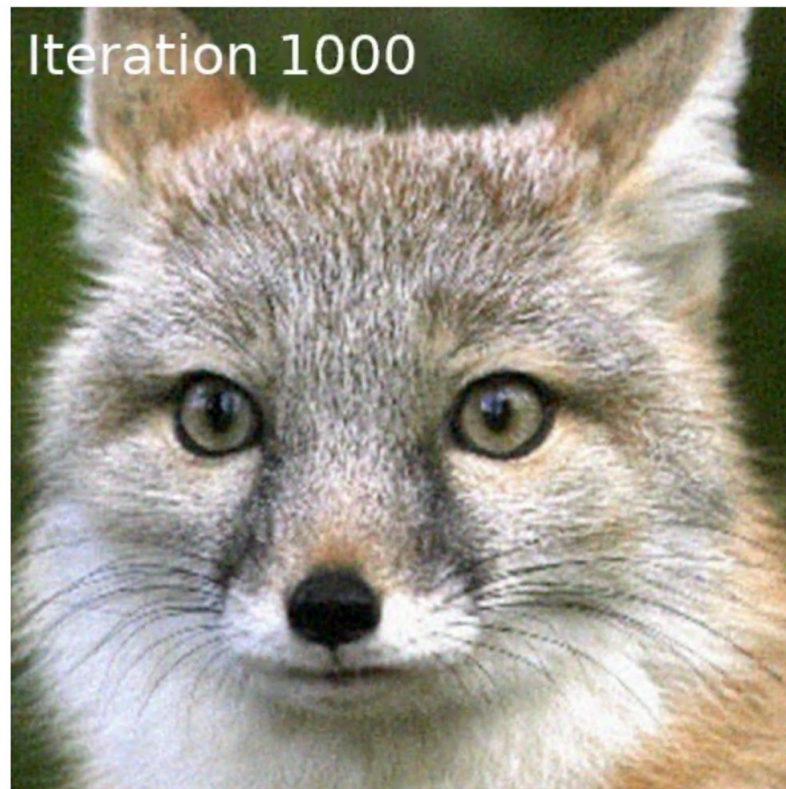
Positionally Encoded input



Positional Encoding

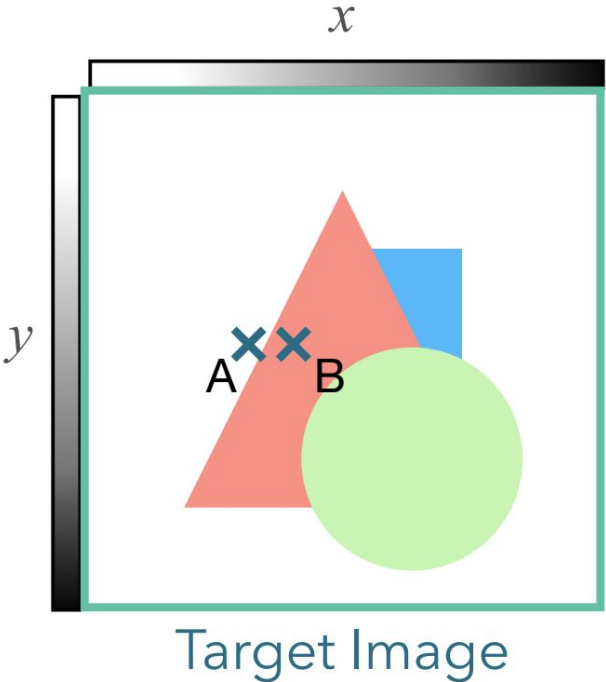
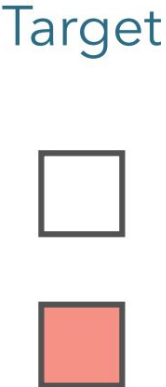
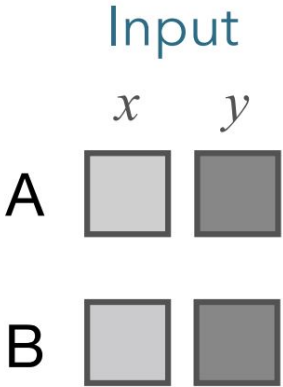


Standard MLP

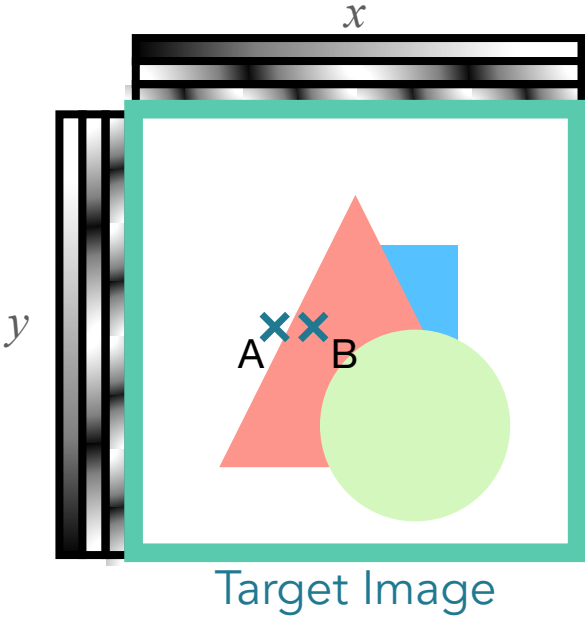
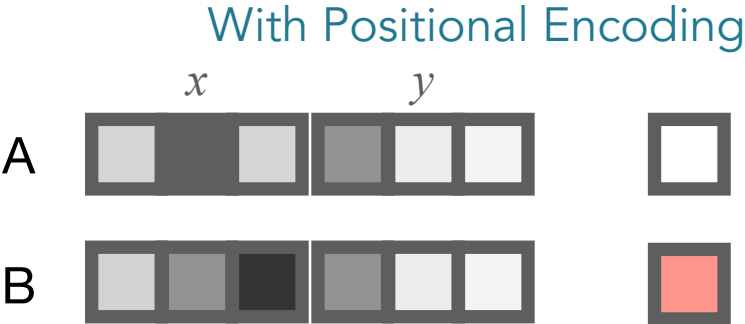
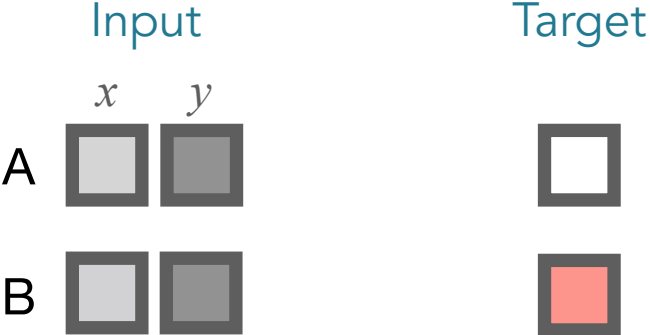


MLP with Fourier features

Positional Encoding



Positional Encoding



NeRF with and without positional encoding

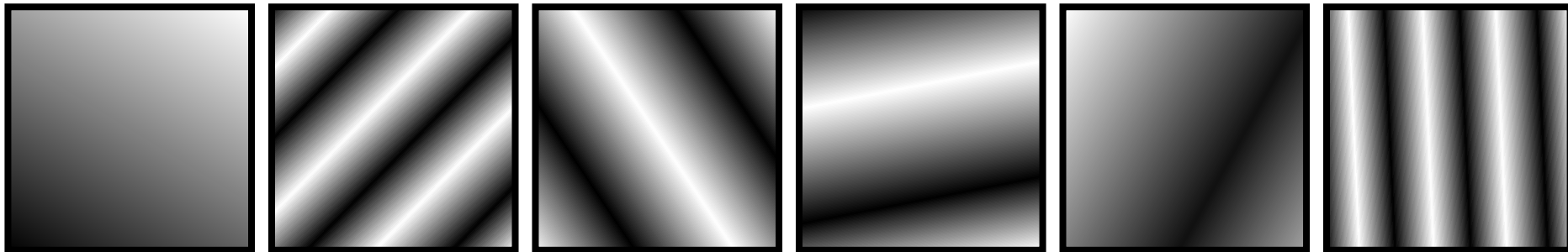


NeRF (Naive)



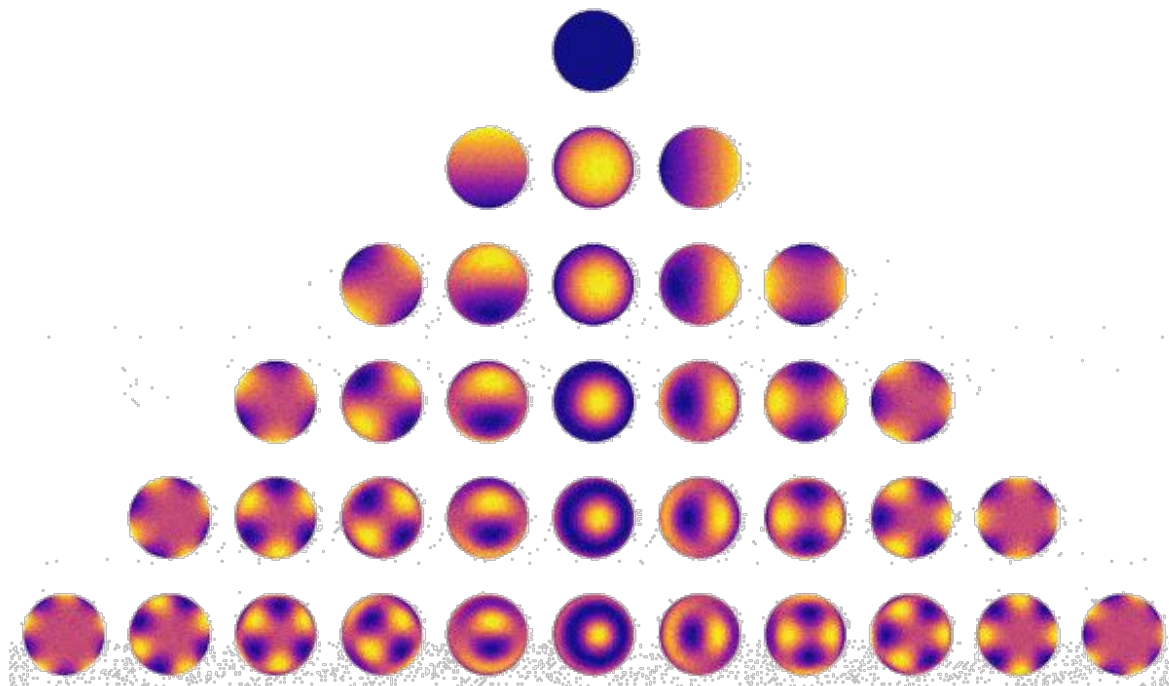
NeRF (with positional encoding)

Other Encoding



Random Fourier Features

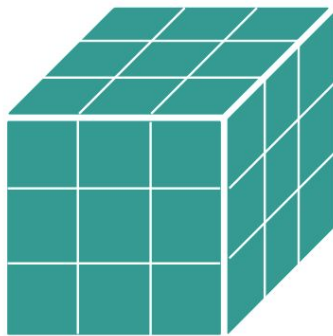
Other Encoding



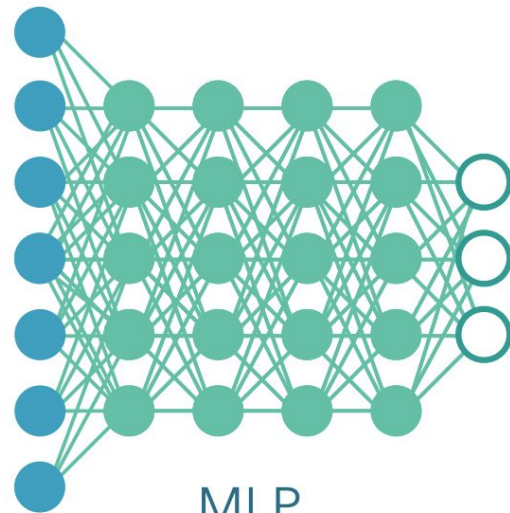
Spherical Harmonics

Other Encoding

Position

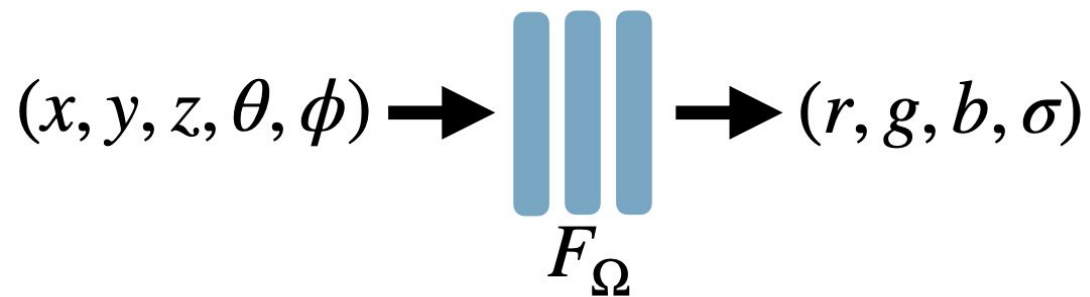


Feature Grid

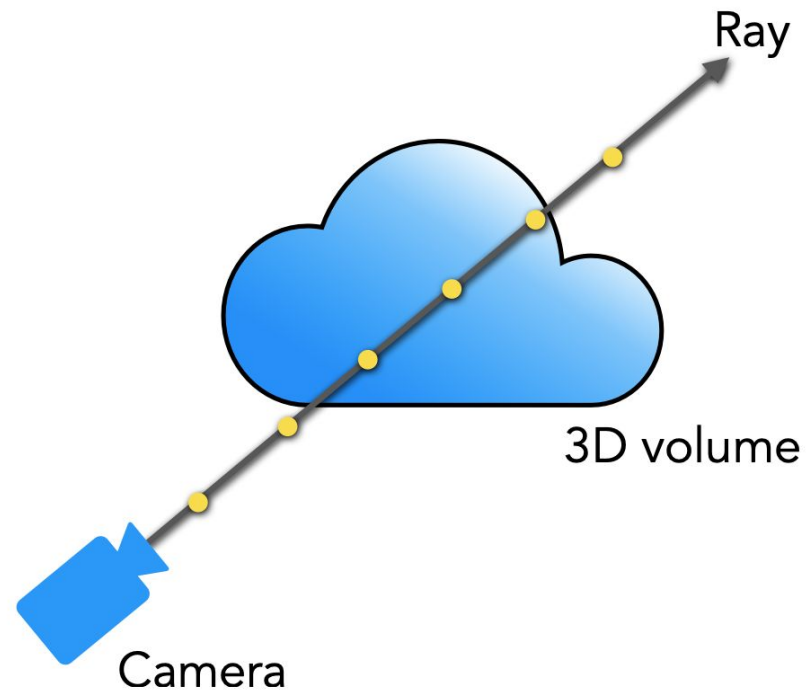


MLP

Neural Volumetric 3D Scene Representation



Differentiable Volumetric Rendering Function



Differentiable Volumetric Rendering Function

$$\begin{aligned} C &= \int_0^{t_{max}} \mathbf{c}(t) \sigma(t) T(t) dt \\ &= \int_0^{t_{max}} \mathbf{c}(t) \sigma(t) \exp \left(- \int_0^t \sigma(s) ds \right) dt \end{aligned}$$

Transparency

Differentiable Volumetric Rendering Function

Rendering model for ray $\mathbf{r}(t) = \mathbf{o} + t\mathbf{d}$:

$$\mathbf{c} \approx \sum_{i=1}^n T_i \alpha_i \mathbf{c}_i$$

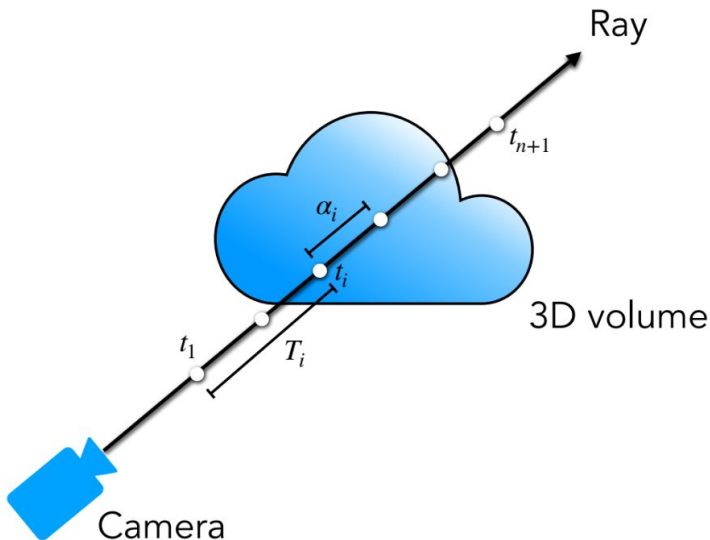
weights colors

How much light is blocked earlier along ray:

$$T_i = \prod_{j=1}^{i-1} (1 - \alpha_j)$$

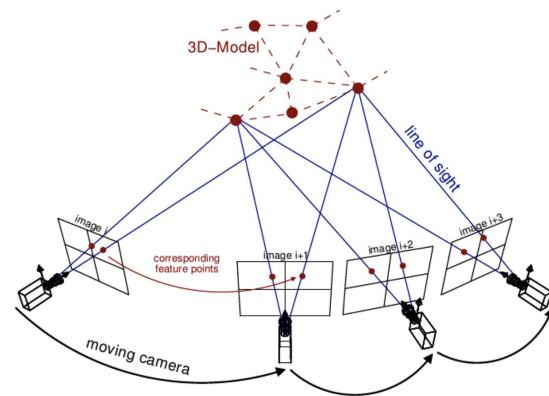
How much light is contributed by ray segment i :

$$\alpha_i = 1 - \exp(-\sigma_i \delta_i)$$



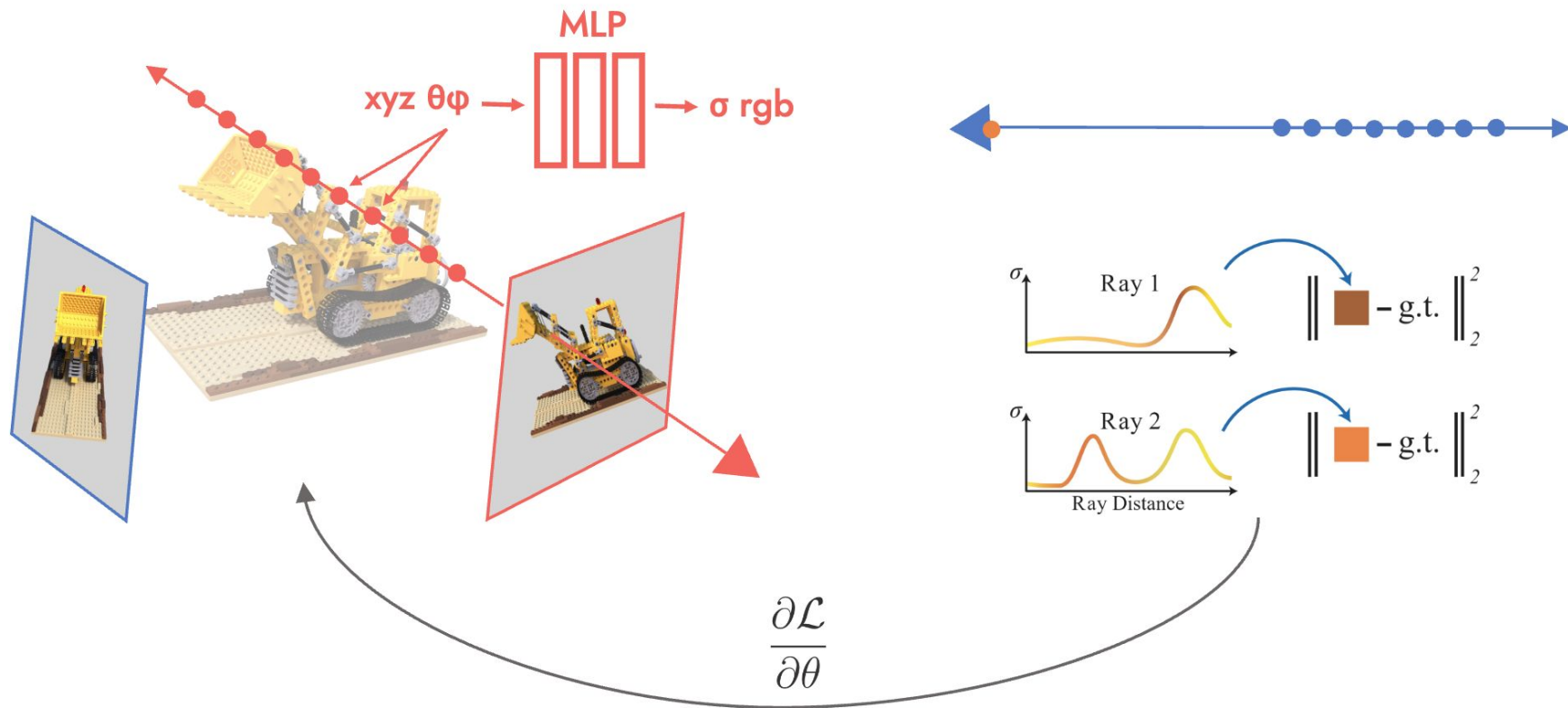
Training

- Need to know the camera parameters: extrinsic (viewpoint) & intrinsics (focal length, distortion, etc)
- In order to train NeRF you need to run SfM/SLAM on the images to estimate the camera parameters



https://www.researchgate.net/figure/Structure-from-Motion-SfM-photogrammetric-principle-Source-Theia-sfmorg-2016_fig3_303824023

Training

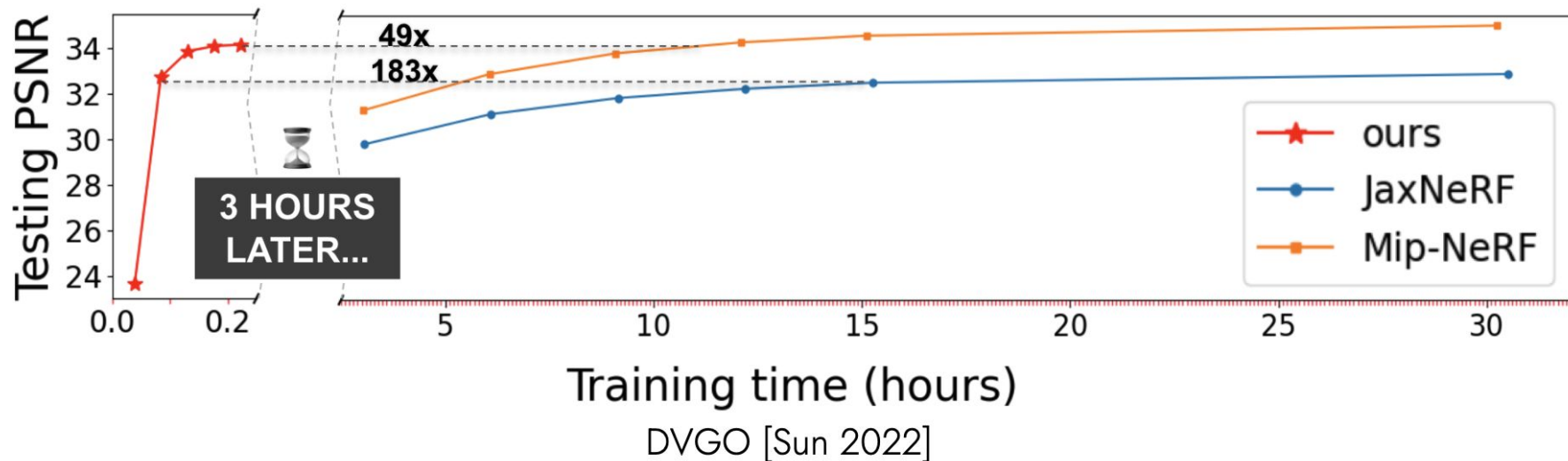


Content of today's lecture

- 3D / Geometric Data
 - Intro
 - 3D representations
- 3D Perception / Understanding
 - Processing
 - PointNet
- 3D Generation
 - NeRF
 - 3D GS

NeRF suffers from slow training and rendering

- DVGO [Sun 2022]
- Instant-NGP [Müller 2022]
- Plenoxels [Fridovich-Keil and Yu 2022]
- TensorRF [Chen and Xu 2022]



3D Gaussian Splatting for Real-Time Radiance Field Rendering

BERNHARD KERBL*, Inria, Université Côte d'Azur, France

GEORGIOS KOPANAS*, Inria, Université Côte d'Azur, France

THOMAS LEIMKÜHLER, Max-Planck-Institut für Informatik, Germany

GEORGE DRETTAKIS, Inria, Université Côte d'Azur, France



Represent a 3D scene as millions of tiny, fuzzy, colored blobs (3D Gaussians), then "splat" them onto a 2D screen to render images.

Ukrainian Translations

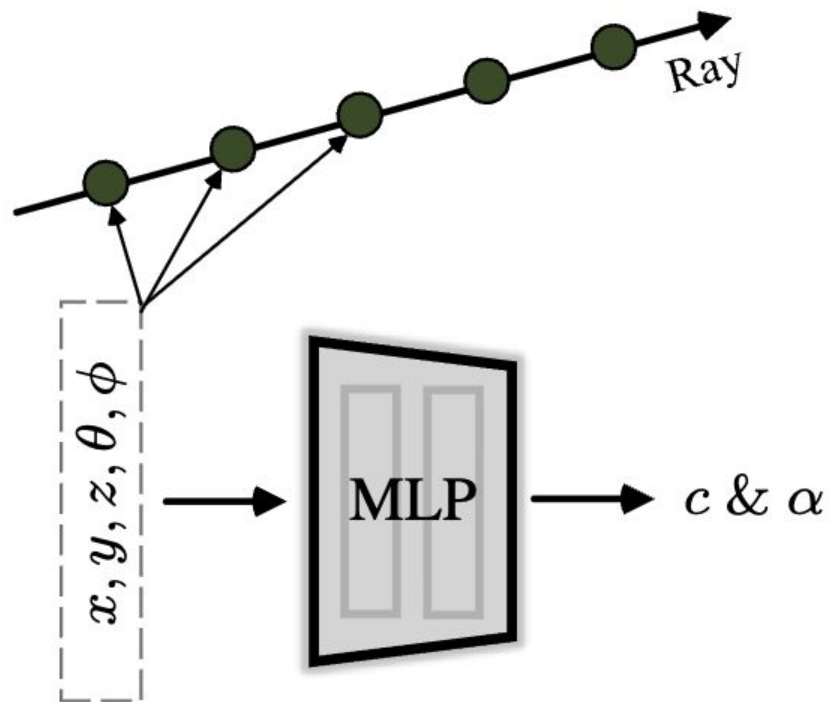
Because "splat" is an imitative and informal word, its best Ukrainian translation often depends on the specific context (what is hitting what, and how forcefully). 

Here are some proposed Ukrainian translations and their contexts:

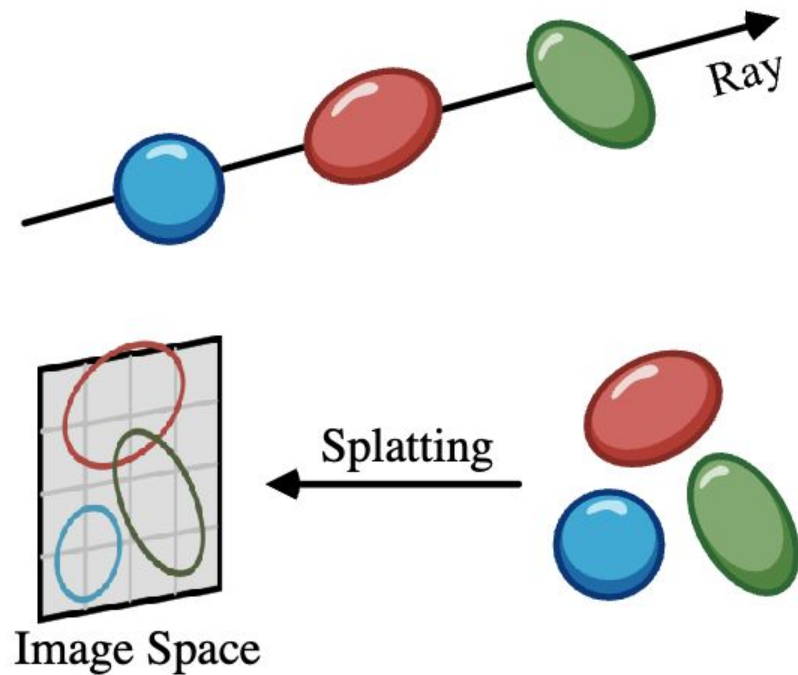
- **Плюх! (Plyukh!):** A common onomatopoeia for a *splash* or *plop*, often of something falling into liquid or a soft, wet landing.
- **Шльоп! (Shlyop!):** Implies a *slap* or a *smack* sound, suitable for a wet object hitting a surface with a flat impact.
- **Ляп! (Lyap!):** Often used for a *blot* or a *smudge*, which can describe the resulting mark or the sound of something messy being applied or dropped.
- **Хляп! (Khlyap!):** Similar to "Plyukh!", this is a general sound for wet, sloppy impact or splashing. 

For general use, **Плюх!** and **Шльоп!** are good equivalents for the English "splat" sound and action. You can use online dictionaries like [Reverso Dictionary](#) for more contextual examples. 

The best and most commonly used Ukrainian translation for the technical term "**Gaussian Splatting**" in the context of 3D reconstruction and computer graphics is "**Гауссове розбризкування**" (Haussove rozbryzkuvannia). 



(a) NeRF



(b) 3D GS

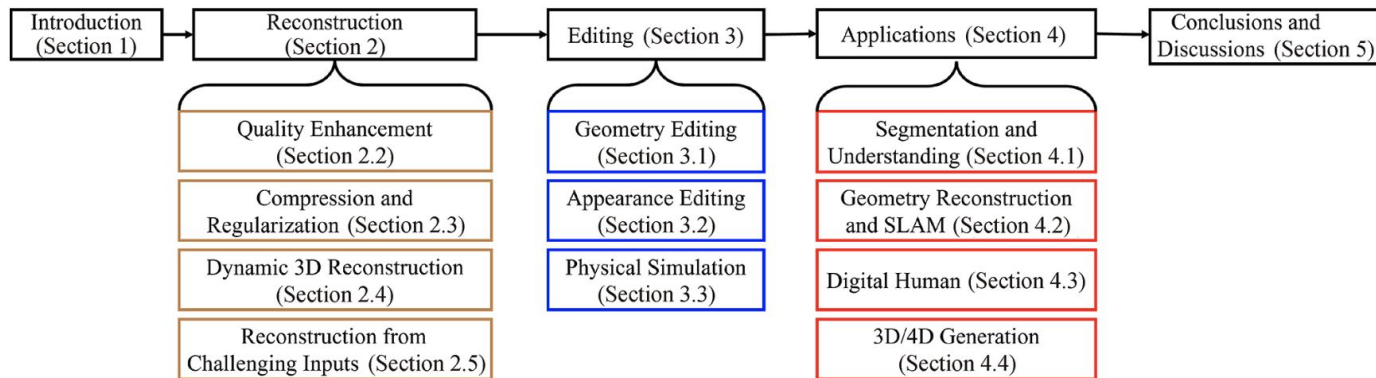
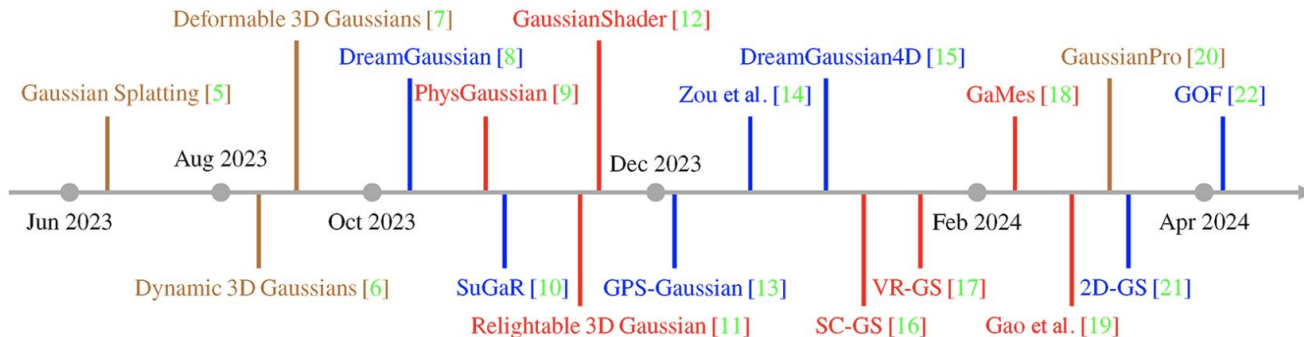


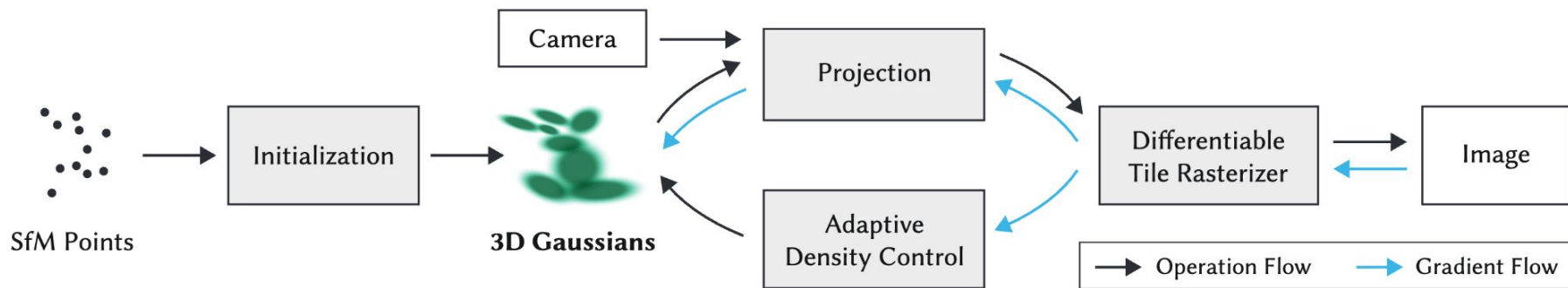
Fig. 1 Structure of our literature review and taxonomy of current 3D Gaussian splatting methods.

— 3D Reconstruction
 — 3D Editing
 — Applications

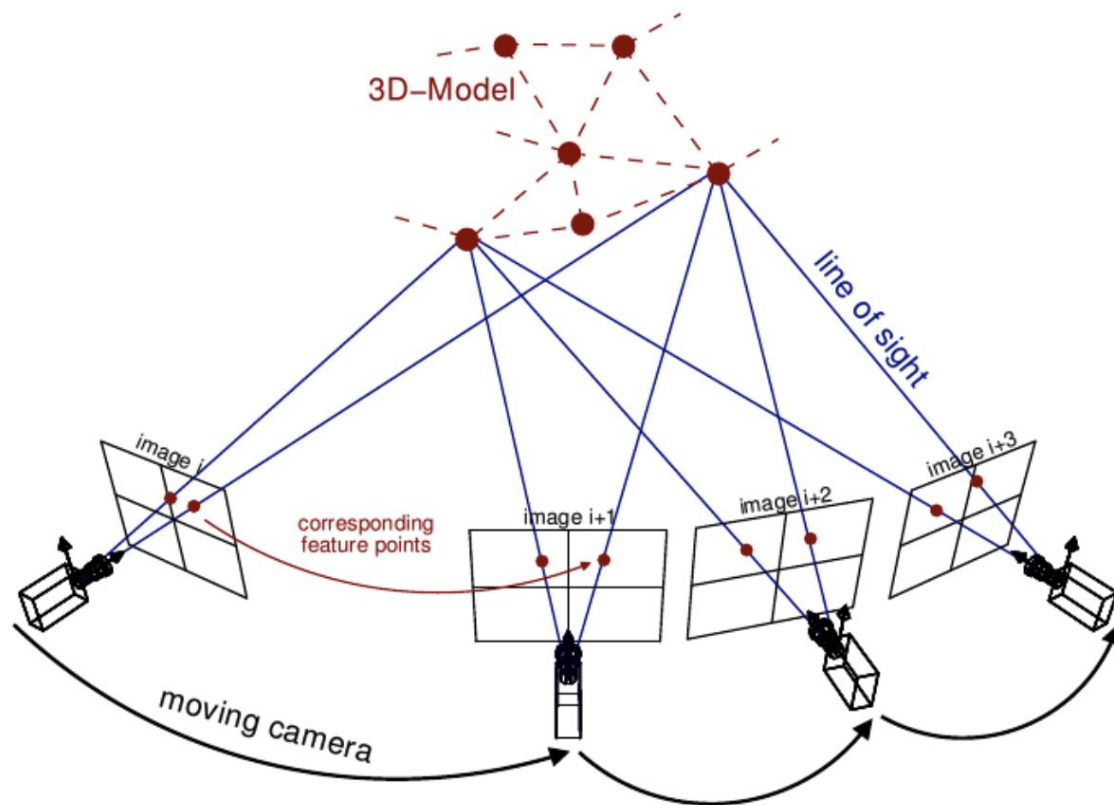


[\[2403.11134\]](#) Recent Advances in 3D Gaussian Splatting

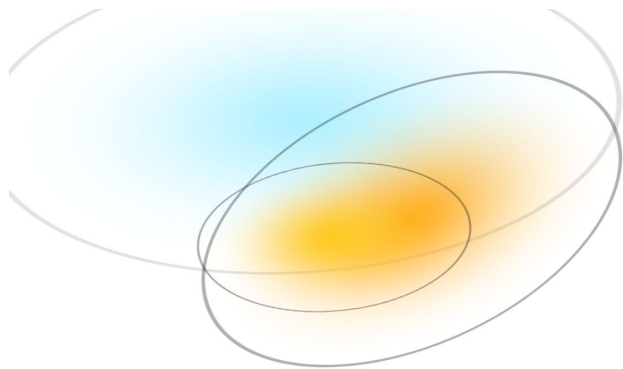
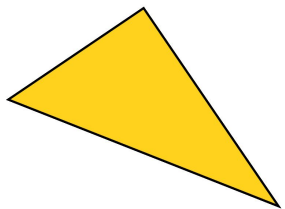
3D GS



3D GS



https://www.researchgate.net/figure/Structure-from-Motion-SfM-photogrammetric-principle-Source-Theia-sfmorg-2016_fig3_303824023



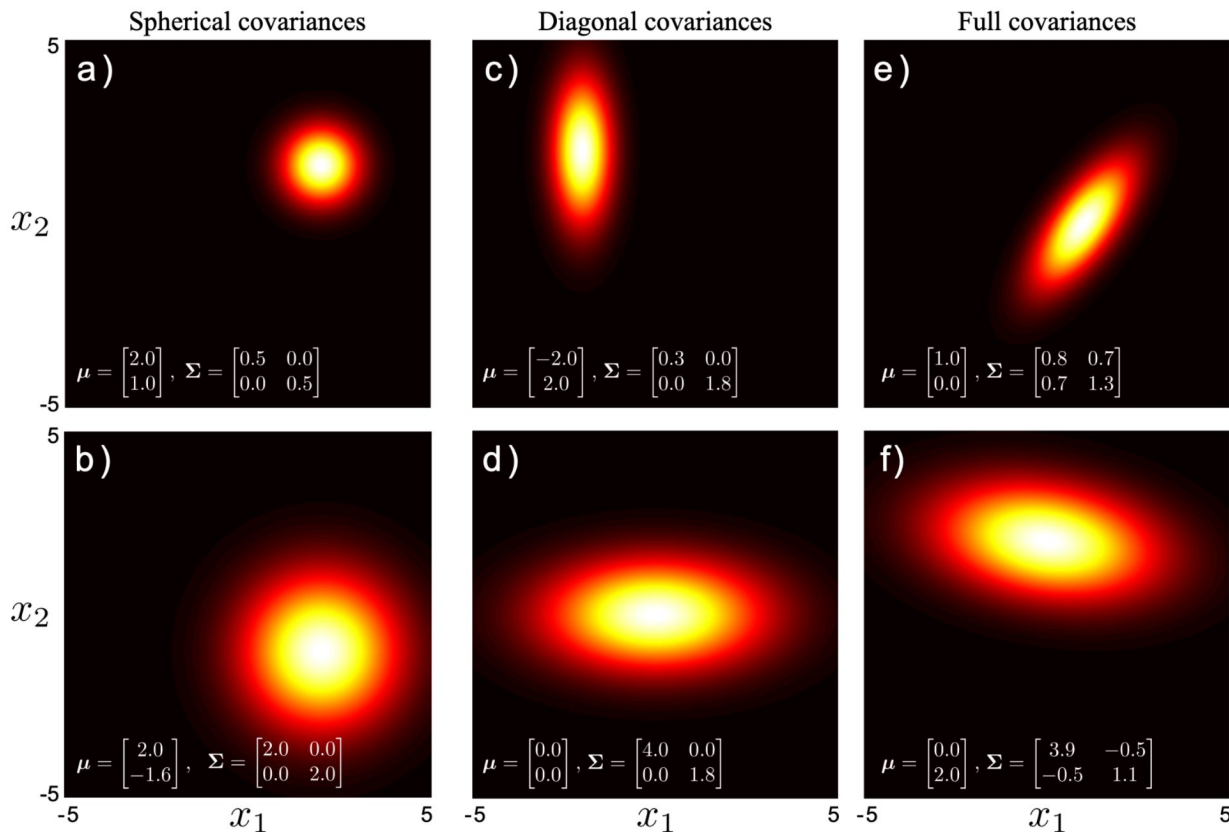
Multivariate Gaussian distribution

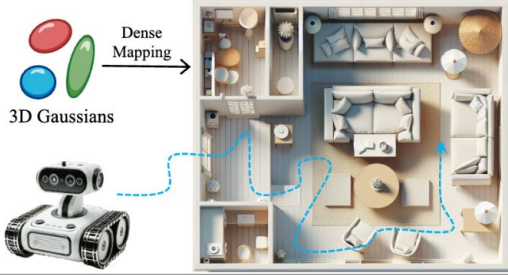
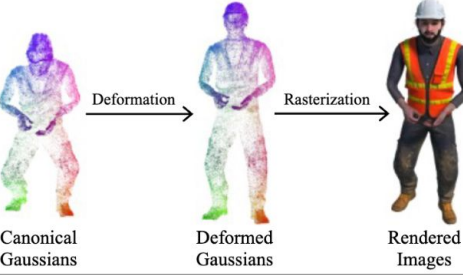
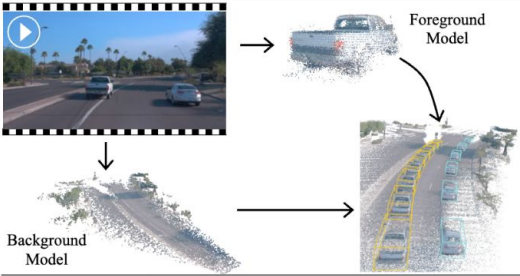
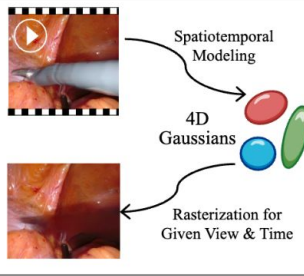
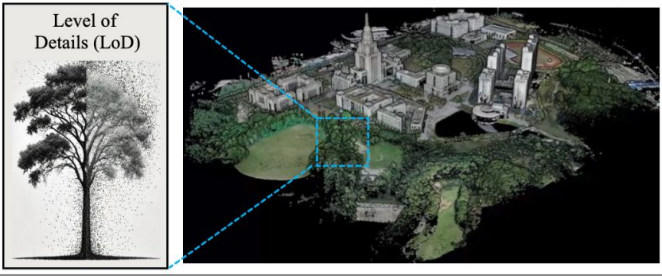
$$\mathcal{N}(\mathbf{x}; \boldsymbol{\mu}, \boldsymbol{\Sigma}) = \frac{1}{|\boldsymbol{\Sigma}|^{1/2} (2\pi)^{D/2}} \exp \left[-\frac{1}{2} (\mathbf{x} - \boldsymbol{\mu})^T \cdot \boldsymbol{\Sigma}^{-1} \cdot (\mathbf{x} - \boldsymbol{\mu}) \right]$$

$\boldsymbol{\mu}$ is $[D \times 1]$ vector that describes the position of the distribution

$\boldsymbol{\Sigma}$ is $[D \times D]$ positive definite matrix
(implying that $\mathbf{z}^T \cdot \boldsymbol{\Sigma} \cdot \mathbf{z}$ is positive for any real vector $\mathbf{z}$)
and describes the shape of the distribution

$$\Sigma_{spher} = \begin{bmatrix} \sigma^2 & 0 \\ 0 & \sigma^2 \end{bmatrix} \quad \Sigma_{diag} = \begin{bmatrix} \sigma_1^2 & 0 \\ 0 & \sigma_2^2 \end{bmatrix} \quad \Sigma_{full} = \begin{bmatrix} \sigma_{11}^2 & \sigma_{12}^2 \\ \sigma_{21}^2 & \sigma_{22}^2 \end{bmatrix}$$



 Simultaneous Localization and Mapping	 Dynamic Scene Reconstruction	 AI-Generated Content
 [104], [105], [106], [107], [108], [109], [110], [111], [112], [113], [114], [115], [116], [117]	 [118], [94], [95], [93], [119], [120], [121], [122], [103], [123], [124], [125]	 [126], [127], [128], [129], [130], [131], [132], [133], [134], [135], [90], [136]
 Autonomous Driving	 Endoscopic Scene Reconstruction	 Large-scale Scene Reconstruction
 [137], [138], [139]	 [140], [141], [142], [143], [116], [144]	 [44], [145], [146], [147], [148], [149], [150], [151], [152]

<https://webgl-gaussian-splatting.vercel.app/>

<https://superspl.at/view?id=ca36efcc>

Content of today's lecture

- 3D / Geometric Data
 - Intro
 - 3D representations
- 3D Perception / Understanding
 - Processing
 - PointNet
- 3D Generation
 - NeRF
 - 3D GS
 - 3D Diffusion Models

3D generative models



<https://huggingface.co/spaces/stabilityai/stable-fast-3d>
[2408.00653] SF3D: Stable Fast 3D Mesh Reconstruction with UV-unwrapping and
Illumination Disentanglement

FEATURED

Computer Vision

Introducing SAM 3D: Powerful 3D Reconstruction for Physical World Images

November 19, 2025 • 10 minute read

SAM3 3D



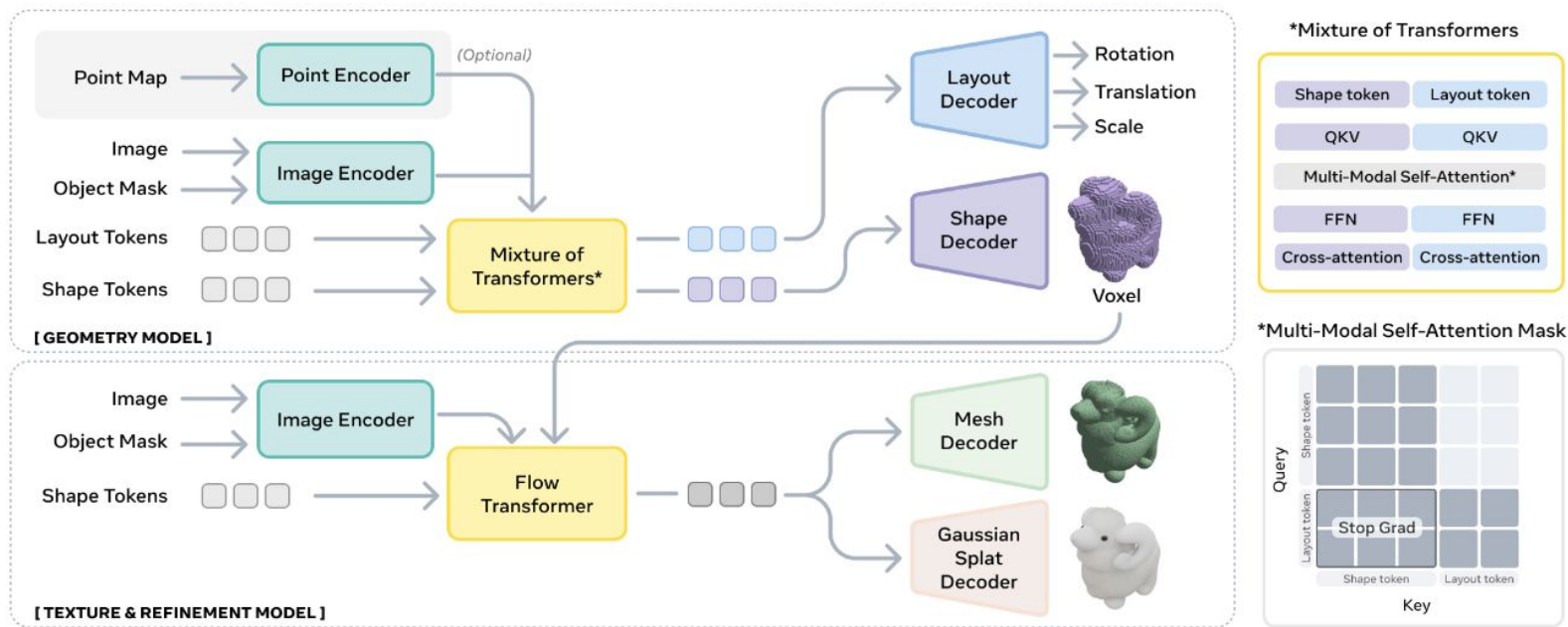
[DEMO: Segment Anything Playground | Meta](#)

[paper: SAM 3D: 3Dfy Anything in Images](#)

[paper: SAM 3D Body](#)

[Introducing SAM 3D: Powerful 3D Reconstruction for Physical World Images](#)

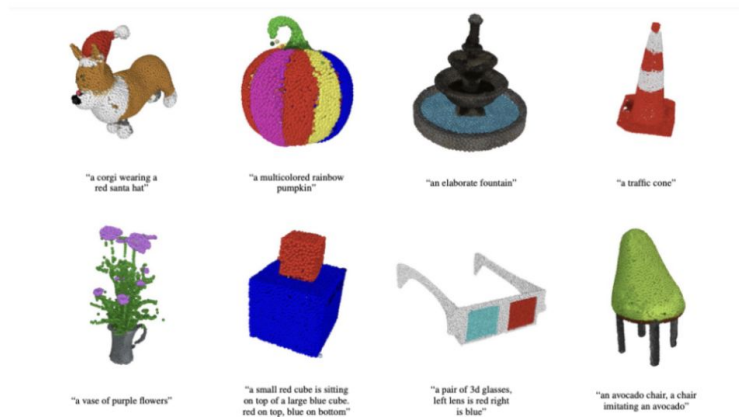
SAM 3D



3D Diffusion Models

Point clouds

- ShapeGF (Cai et al., 2020)
- DPM (Luo and Hu, 2021)
- LION (Nichol et al., 2022)

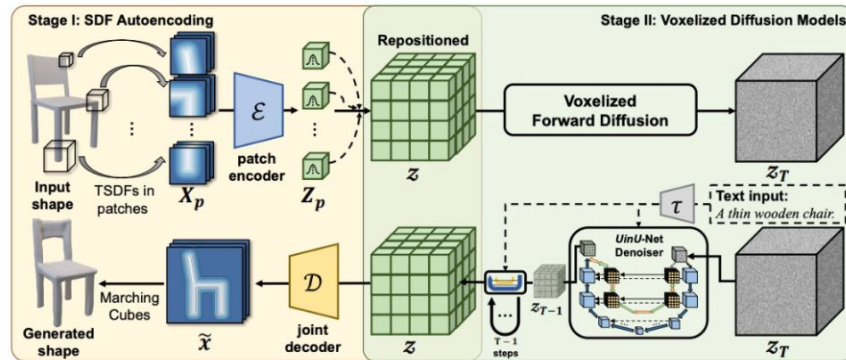


LION, Nichol et al. 2022.

3D Diffusion Models

Voxel representation

- PVD (Zhou et al., 2022)
- Diffusion-SDF (Li et al, 2022)

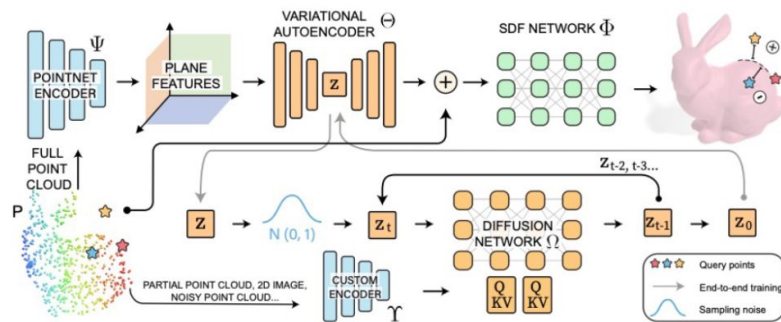


Diffusion-SDF, Li et al., 2022.

3D Diffusion Models

Latent representation

- SDFusion
(Cheng et al., 2022)
- DiffusionSDF (w/o *hyphen*,
Chou et al., 2023)
- 3DShape2VecSet
(Zhang et al., 2023)



DiffusionSDF, Chou et al., 2023

Other materials

- Courses

[CS231A: Computer Vision, From 3D Perception to 3D Reconstruction and beyond](#)

[CS479 Machine Learning for 3D Data](#)